Linux - 6 :- Linux system programming

Topic :- Introduction to O.S

1) what is OS ?

2) why os in Embedded system ?

3) why linux.

4) what are the components of os Random number generation rand(), srand()

1) what is OS ?

operating system is a software, which is running on the top of the Hard-ware. under

* OS is not a Hardware

Non-technical defination for operating system.

it is the mediator b/w the user and hardware.

Technical defination for operating system.

OS is a Resource Manager. Here resource is nothing but all the hardware components → Eg :- RAM, Rom, Hard disk, monitor, keyboard, Pen tab Bluetooth, wifi-module etc.

* operating system is responsible for managing all these hardwares. *

* without operating system, we can't imagine the laptop (or) we can't imagine the desktop.

* without operating system, it is very difficult to manage, All these Hardware components *.

* Now a day's in embedded system also, operating system involvement will be there.

Previous days → Embedded System "means" which can perform only one tas But now a days embedded system means which can perform multi-tasking.

* Now a days most of the Embedded products are coming with in bu ~~Embe~~ O.S

* The os which is Embedded with in that specific Hardware. especially linux os.

* in our mobile phone also having o.s, because of operating System only, we are able to do multi-tasking.

* linux is a free source code, Linux can be loaded into different different hardware architectures * (Even in small small micro controllers also).

* where as os can be loaded into only 2 i,e intel and Arm.

*Reasons for why linux is popular:-

1) Free source code and open source operating system.
2) we can port it into different, different hardware Architectures.
3) There is no propretary ship
4) The source code can be modified and distributed to anyone commercially or non commercially under the GNU (General public license) commercially or non commercially under the GNU (General public license)
5) High security, large community support, high stability and higher performance.

Embedded linux :-

* Embedded Linux is a type of Linux operating system | kernel that is designed to be installed and used within embedded devices and applications like,

---- Consumer electronics (set-top boxes, smart TV's, mobile phones)
---- in-vehicle infotainment (IVI)
--- Networking equipment (such as routers, switches, wireless access points or wireless routers.) --> so many Advantages of linux o.s

Strong point :-

"when os is there, multi-tasking can be possible."

** distinguish b/w o.s and cpu.

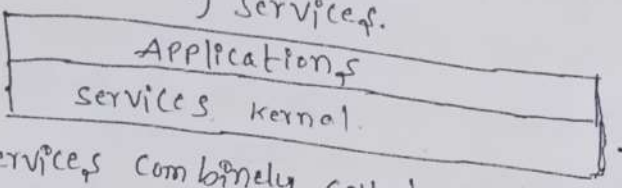
-> cpu is a hard-ware while os is a software.

Imp Note Point :- when we are switch on the system, our os should be loaded into Ram for the execution.

What are the components of operating system. ?

majorily, the operating system contains, two components.

They are:- 1) Applications.
2) services.



→ services/managers ✓

* All the services combinely called as kernel.
* Kernal is a part of an operating system
* kernal is a core part of an operating system.

→ In this kernal all the managers will be there, like 1) process manger, which is responsible for managing the processess.
2) memory manager :- which is responsible for managing the Ram,
3) File-manager :- which is responsible for managing the Hard-disk.
4) Net-work manager :- which is responsible for transfering and relieving the data from the network.
5) Thread manager :- which is responsible for managing the thread.
6) signal manager :- which are responsible for managing the signals.

→ All the above managers are present in kernel space.
→ All the managers are combinely called as kernel.
→ "kernal is also present in windows"

in simple words kernal is nothing but managing different different tasks. Some times files. [Above, all the managers are nothing but kernel].

services are capable of communicating with the Resources.

statement :- There are so many applications are there, to create a file. like by using vi editor, Paint app, Note pad, word pad..etc by using all these applications, we can create a file.

All these are Applications → who uses these applications? → user.....

** → what these applications will do inside ?

Ans → communicates with kernel, communicates with OS. (main component). and these applications are communicate with file manager.

These managers are responsible for communicate with hardware.

imagine, if there is no file manager, then there is no use of applications.

So, that's why services are mandatory.

* Question :- Applications are mandatory (or) services are mandatory?
Ans :- services are mandatory [explaination already there in previous page. below also :-

Reason :- if we are not having one application for particular task, we can use another application for same task.

Example :- I want to browse, so even though I am not having chrome I will use fire-fox for browsing.

case 2 :- I have many applications, but no internet service or network manager is not there then useless.

case 3 :- As we already discussed, we can create a file using notepad vi editor, XL, word etc. → if word or notepad not there, i can use XL for creating a file. but if file manger (service) is not there then use-less.

Note point :- Some of the services, when os is installed, automatically services are also installed.

But for some of the services you need to install explicitly.

→ Now a days 90% apps need internet in our phone.

Final conclusion :-

Applications + services combinely called as O.S

Note :- Some times we are getting notifications in our mobile as update app. Also rarely coming → update os (system update).

→ what is the meaning of system update :- (or) os update :-
simple justification → As we know that, os is a combination of servicy & applications.

→ so System updation "means" may be there is a bugs in manager, for fixing that bugs, we have to update it.

Task to the student :- you guys know, that we are using Android os in our mobiles, now your task is, you have to find, which kernel version your using - by Entering into Android settings.

Distinguish b/w shutdown and Restart, ?

Very simple, The name itself shutdown means, completely power off. Restart means, Restart Again -> but important point need to consider -> some times, we can observe, After updating the system/os it is Asking update & shut down (or) update and Re-start. when update and Restart then only old managers (which are having bugs) Eliminated and new managers are installed.

our main target in this module is we are going to communicate with these managers by using system calls.

↓ by using Programs only we are going to communicate. :-

Linux-7 :- Booting steps of linux os :-

Question :- How much time, it will take, After Power system, to get login id password interface?

Ans/ it depends on operating system, Maximum 1 min to 2 minuites.

Now, we are going to talk, what is happening b/w that 1 min gap. (or) what is happening in the Back-ground. that is called

→ **Booting steps :-**

when we switch on the system, the login screen comes, in back-ground there are 6 -> steps takes place ->.

- -> BIOS (Basic input/output system Executes MBR
- -> MBR (Master Boot Record executes GRUB)
- -> GRUB (Grand unified Boot loader Executes kernel
- -> KERNEL (Kernal executes /bin or init.
- -> INIT (Init Executes Run-level Programs)
- -> RUN Level (Run-level programs are Executed from /etc/rc.d/rc*.d/

In this bios settings only we need to set the priority of booting, from where o.s will be loaded → 1st priority, 2nd, 3rd -- like this.

step-1 Bios :- Note :- this Bios program is present in the mother_board in the Rom chip in the mother board, Apart from the Ram and Hard_disk manufactures of the cpu, they will put one chip in the mother board that is called Rom chip. (In this chip only Bios program will be there.

Conclusion :- so, when we switch on, laptop or desk_top, the first program, that going to run is Bios Program (which is in Rom).

Points to be Noted under Bios → step-1 :-

* Bios stands for Basic input/output system.
* Performs some system integrity checks.
* Searches, loads, and Executes the boot loader Program.
* During the Bios startup to change the boot sequence.

Detail Explanation of Above steps :-

when we are using the cpu, & when we switch on the cpu, Bios Error, we will get. [in Blue screen we will get Bios Error].

→ the Responsibility of Bios program is, the basic input/output Peripherals are properly inserted (or) properly connected (or) properly working or not it will check. → like to run the system Ram is very important, to run the system mother board is very important, to run the system Hard-disk is very important.

So, the Bios program will check these devices (Ram, mother board, Hard-disk etc) are properly connected or not.

→ it will send some signal and waits for ~~signal~~ reply, if reply came means, that device is connected properly.

→ Is monitor is a important device for loading an o.s with out monitor operating System loaded or not → Ans → yes loaded.

→ monitor is not an important device, but Ram is important bcz os must and should be loaded into the Ram.

→ will you agree, Hard disk also loaded → ? (doubt)

So, what this bios program will do means it will Communicate with those devices [don't think how they are communicate, they are having some own set of Rules and Regulations to Communicate].

while communicating bios program wait for the reply, if reply not came, so simply it will give Bios Error.

interesting Point:- if dust particles are Accumulated on Ram, Slot then also Bios Error will come, that's why network Administrators are cleaning the Ram slot by removing it and they will Again insert it.

question:- in which Program this bios program will be written?

when we go for the low level it is written in Assembly.

Ami → American mega trends, they will work on the Bios program.

2nd point:- performs some system integrity checks. (All the Basic I/P O/P peripherals are properly connected or not it will check.

3rd point:- And also searches, loads, and executes the boot loader program.

↓ we know that when using command ./a.out → Enter, our app is loaded into the Ram for Execution → similarly somebody needed to load operating system into the Ram.

where operating system (O.S) is present?

when O.S is installed, it is stored in Hard-disk (secondary memory) from the Hard disk it should be loaded somebody and also through pen-drive Also we can boot O.S.

In this Bios_bootable pendrive, we can change the priority also from where the os should be fetched like →

→ is it from the pendrive?
→ is it from the Hard-disk?
→ is it from the CD Rom?
→ is it from the Network?

Now a days, the operating system throug network also we can boot.

"manufacturers of the mother board will load this bios Program into the Rom chip!!

4th → we can change the booting sequence also (Bios program) booting sequence means from where the os is booted or loaded —

→ Bios will give control to the MBR.

↓

STEP 2 :— MBR (MASTER BOOT RECORD)

that bootable disk may be pendrive or hard_disk

- MBR stands for Master Boot Record.
- it is located in the 1st sector of the bootable disk.

Ex:- when we are talking about, hard_disk → we call "term" called Sectors.

* MBR is less than 512 bytes in size.
* it contains information about Grub. (Loader information)
* so, in simple terms MBR loads and executes the GRUB boot loader.
* MBR is not a program, it is a file/Record which is present in the starting sector of our bootable device. [in this starting sector, Grub information will be there.

STEP 3 :— GRUB (Grand unified Boot loader).

. (Every operating system has its own loader)
(Windows operating system loader name is NT Loader)
(Linux operating system loader name is LILO loader.

1] GRUB stands for Grand unified Boot loader.

* one of the speciality of this loader is, by using this loader, windows can be loaded and linux also can be loaded.

* Every operating system (software) designers, they will create their own loaders.

* in our lab also some systems are having windows as well as linux (Dual booting).

↓

→ when we switch on the system, it will display one screen, in that screen, windows will be there and linux will be there. and if systems not working we will observe Grub Error

so, speciality of Grub is it can load multiple operating systems.

→ In linux also, we can put multiple kernels (multiple managers)

→ operating system loaded means", the managers are loaded into Ram for execution.
who will load kernel into the Ram. ?
GRUB will loads. → if we have multiple operating systems, then Grub will display which operating system you want to display ? And also when it display time also displayed down and going in decreasing order → like 30, 29, 28, 27 ... 1 (observe in lab (or) Asle the Admin).
*so depends upon our choice, the Grub will loads the kernel to the Ram for the Execution.
* Kernal is present is hard disk ↓
→ windows operating system is present in the C-drive. but linux os is present in sbin.
so, After loading the kernal, what kernel will do means, it will execute one program which is there in sbin (i.e init).

step-4. KERNEL.

① Before the kernel is loaded by GRUB, kernel mounts the root file system.
② eg:- Ext2, Ext3, FAT, FAT16, FAT32, NTFS file systems.
"without file systems there is no meaning of data, which is present in the Hard_disk."
"one of the speciality of linux is, linux will understand all other file systems also.

Question ? In a pen-drive which file system is present ?
Ans) FAT file system, FAT 32 (BC3 of these file system only we are able to communicate with pendrive.
* Every operating system has its own file system.
linux file systems → Ext2, Ext3, Ext4 file systems.
windows → NTFS.
without a file system, there is no meaning of the data which is present in the file in a hard disk.

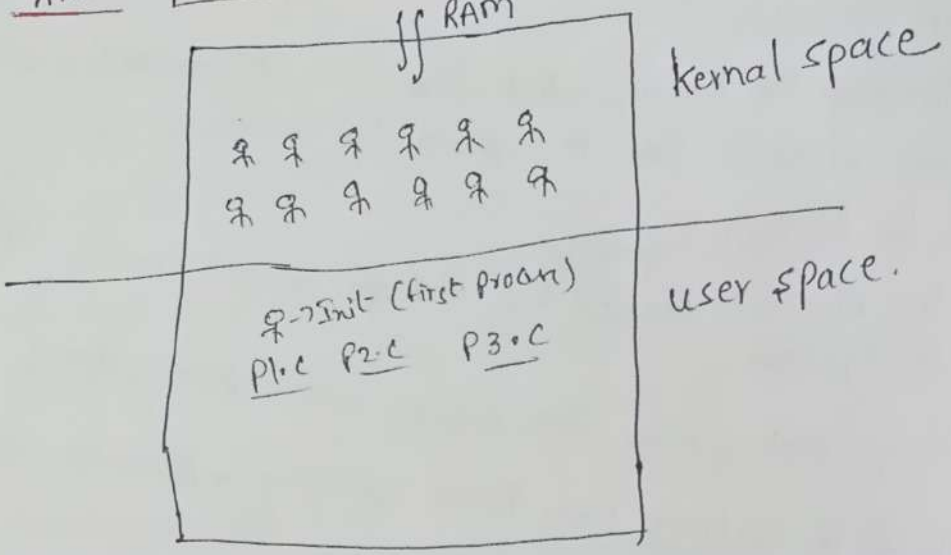
step 4: kernel

* mounts the root file system
* kernel executes the /sbin/init program.
* in simple terms we can say os is loaded into Ram.

step 5: INIT:-

* since init was the 1st program to be executed by Linux kernel, it has the process id (PID) of 1
* init identifies the default init level from /etc/inittab and uses that to load all appropriate program.

RAM SPACE IS DIVIDED INTO 2 PART S :-



→ Ram is divided into 2 parts → (1) kernal space, (2) user_space

can you tell me who is there in kernel space?

In kernal space, all the managers are present, eg: process manager, memory manager, file manger, Network manager, signal manager, memory manager etc.

can you tell me, who is present in the user_space?

our Applications, our programs → our programs will run in the user space.

Note:-

so, once file system gets mounted and once the kernel is loaded into the Ram, the first program that is going to be executed in user space is init. Process. This process runs, till we power off the system.

"the programs which are running in the user space they are called as technically called as processes!!

"the programs which are running in the kernel space they are technically called as services!!

"with respect to O.S, services are also programs which are executing but services are not an application, they are services which provids a service to an Application.

our next few sessions/lectures our aim is to Communicate with the services by writing code → "means" we are going to create a processess that are going to Communicate with services.

Note :- when we intall windows OS some services will be there, some applications also there (By default it will come).

Note :- Applications are started Executing, whenever we want As we know in C. language. by layout or by clicking on App.

But → services are started Execution, when the operating system is loaded immediately after the switch on the power.

"Applications when they are in Ram, internally communicates with services.

*if no proper service "means" then there is waste of using an Application.

Note :- All the processess are under the control of init process.

what init will do "means" all the small, small services it starts Running.

init will Executes some of the small, small programs → eg :- smTP, (simple mail transfer protocal), TFTP, FTP (File transfer Protocol) all these are started Running at Run level programs (who will Executes/starts these services means → init will start.

And Also in this step only one important point is decided at which run level our O.S should run. → following are the available run levels :-

0- halt
1- single user mode → eg my laptop
2- multi user, without NFS. → eg lab.
3- full multi user mode.
4- un-used.
5- X11
6- Re-boot

if Everything is done Smoothly then login screen comes.

STEP- 6 RUN LEVEL PROGRAMS :-

* when the linux system is booting up, you might see various services getting started, for example, it might say "starting sendmail ... ok" those are the run level programs, executed from the run level directory as defined by your run level.

* in simple term we can say small services start executing in this step. and decides linux run level.

All above 6 steps will read in RTO's concept further in detail.

---

linux_8:- Topic:- process Management :-

We have to discuss following concepts in this session:-

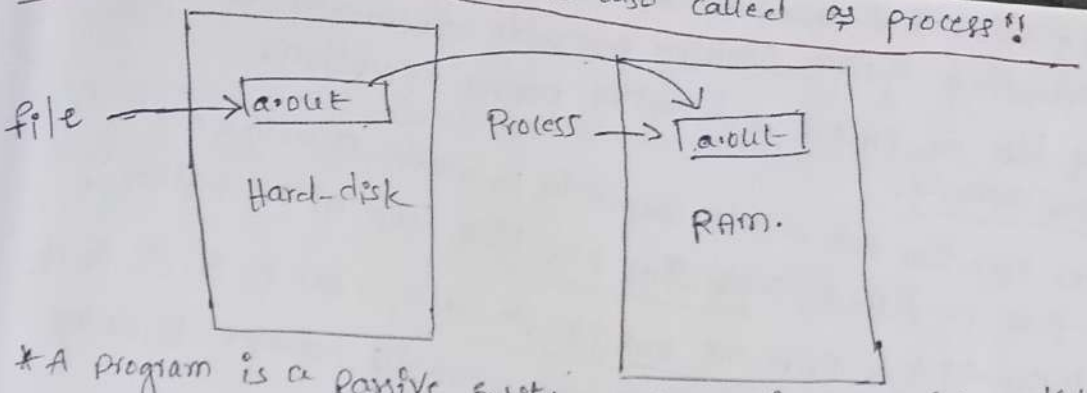
1) what is process.
2) what is multi-processing. How many types.
3) what is process management.
4) foreground and Back-ground process in terminal.
5) Basic Commands in process Management.
6) Memory layout of process
7) what is shell.

1) what is process ?

A process is nothing but A program which is in Execution is called process.

* process is instance of program in execution. what ever data present in hard disk is technically called as file. if that file has capability of getting executed, then it is called as program. copy of program, present in the Ram is called as Process.

"Application Active in Ram are also called as process"!



* A program is a passive Entity, such as file containing list of instructions stored in Disk.
* where as process is active Entity.
* A program becomes a process when an Executable file is loaded into Memory.

+ size of Executable file (.Exe) is less when it is in Hard disk than the same Executable loaded into Ram, i.e its process is created.
Small reason → .Exe → [data | text] → but in Ram stack & heap also present.

* life of that file in Hard disk is permanent while life of process is till it completes its Execution.

Example Code-1

```
#include<stdio.h>
int main()
{
  printf("Hello\n");
  while(1);
}
```

in above Example, process not terminated we are forcefully terminating the process by using command Control-C!

Above program is never going to complete, if it is in while(1) loop.

Example Code 2:-

```
#include<stdio.h>
int main()
{
  printf("Hello world\n");
  sleep(10);
}
```

→ The Above process will be terminated after 10 secs automatically.

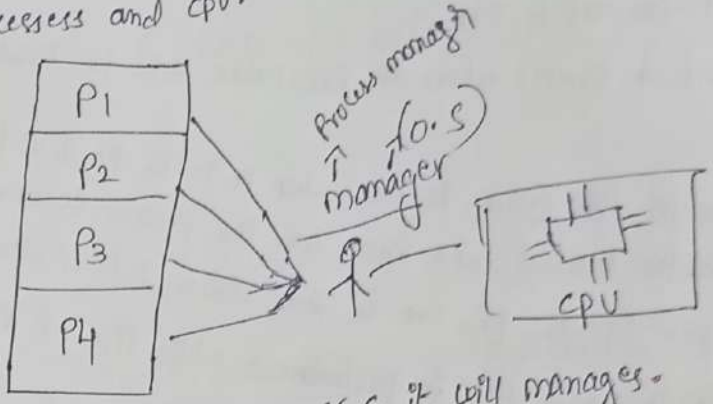
↑ before 10 seconds also we can terminate this process by pressing Control-C. Here by pressing the Control C - we are interupting the process, we are sending one signal to the process.

In our mobile also multiple processes can run

Interesting Question :- when multiple processes are running after Completion of 1 Process then only next process executes or before Completion of 1st Process only 2nd process is executing.

Ans) Before one Process is Completed, Another Process is Executing. which is called As Concurent Execution, Concurently both are Executing.

* "Linux operating system & windows operating system is a multi-processing operating system. (multiple processes concurrently can run)
No dependency b/w one process and another process execution.
concurrently / simultaneously.
→ In case after executing one process completely, then only executing next processess then it called sequential execution (one after another).
→ Now logically think → there are multiple processes → like P1, P2, P3, P4 and one CPU, so there should be a some responsible manager, to manage these processess and CPU.



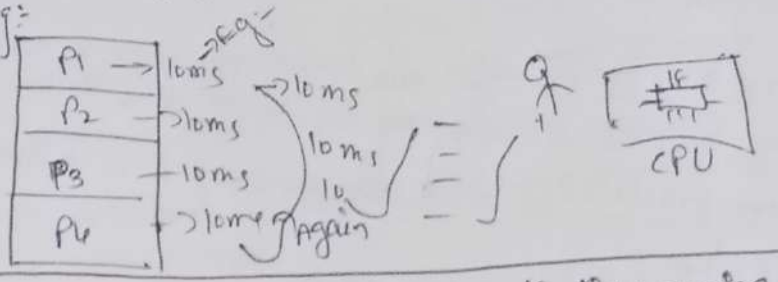
Here, operating system is a manager, so it will manages.
Question ? → only one CPU is there, and multiple processes are there to execute, then who is responsible for managing it ?
Ans) → process manager.
Actually, user thought, multiple processes are executed by CPU at a time.
→ But Actually, only one process instruction can execute by one CPU. (that much faster switching happens)
→ The process manager job is to process the manager and CPU.

Question :- Why they need to manage ?
Ans) Because only one resource is there, multiple processes want that resource, so this resource should be distributed to these processes.
very interesting → All the managers are present in Kernel space Eg :- network manager, file manager, memory manger), so similarly process manager also in Kernel space.
mind blowing point :- when listening the music, and chatting in what's app, user feel that both music player and what's app executing at a time.
but practically, when we think only one can run at a time. but that feel we will not get. It's the job of the process manager, user can not feel that, now P1 executing, now P2, now P3. like this

Actually user feel that Every one is Executing. (Bcz that much faster switching happens → within micro secs switching).

"process manger will Allocate cpu time to Every process micro secs → so that switching we can't feel.

Eg:-



2) what is multiprocessing :- multiple applications running simultaneously or concurrently are called as multiprocessing.

* Benifit of multiprocessing is execution of one application is not dependent on other application Execution

Eg:- Linux and windows are multiprocessing operating system where as Dos is single processing system.

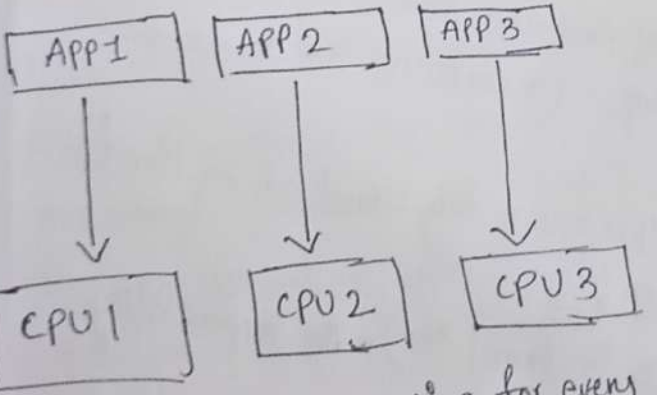
* multi-processing can be achieved in two ways:

1) Hard-ware multiprocessing :- simply known as multiprocess

2) software multiprocessing :- simply known as multiprogramming.

multiprocessing :-

Hard ware multi processing



→ in Hardware multiprocessing, for every application there is a dedicated CPU i.e called Hard ware multi processing.

→ in which Environment Performance is too high?

obviously, in hardware multiprocessing bcz for Every Process there is dedicated CPU there is no disturbance;

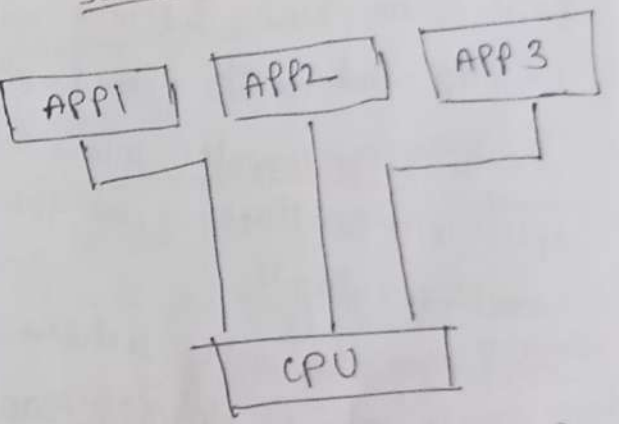
* cost is high required more CPU.

no of APPs & no of CPU is also too high.

Cost of Product is also too high.

(57 mins)

software multi processing :-



in software-multiprocessing one CPU is shared by multiple processes.

→ little bit delay here, so Performance is bad compard to Hard ware multiprocessing.

for suppose, I want to design one embedded application, in that application for suppose i need to execute 100 processess, then i need 100 cpu → in case of Hard-ware multiprocessing. → so cost of the product becomes too high.

Small Note:- (1) Quad core Processor:- Quad-Core processor is a processor which is having 4 cpu's. → 4 instructions can execute at a time.

(2) octa-core-Processor:- octa-Core processor is a processor which is having 8 cpu's.
8 instructions can executes at a time

is quad core processor and octa-core-processor belongs to Hardware or Software multi-Processing?
Ans → still they are software-multiprocessing only → why bcz more than 8 applications are running in the back-ground. and there Among 8 cpu's. one cpu is going to shared by multiple applications.

(B) Dual-core-processor:- Dual Core processor is a processor, which is having 2 cpu's. → it is also Software multiprocessing because, each ~~processor~~ is cpu is going to share multiple ~~applications~~ processess.

Most of the places → "Software multiprocessing only". → eg:- our lap top, Desk top and Mobile phones → etc.

cosider, super computer, where for every processes, there is a separate dedicate cpu will be there → chat gpt saying super-computers are Hardware multiprocessing!!
→ Servers also having software multiprocessing only most of the times.!!

What is Process management

*process management is one of the service in kernel (os). its responsibility is to manage all the running processes of the system.

* Comparing to Hardware multiprocessing, in Software multiprocessing Process Manager's role is very important. Because one cpu need to shared to multiple processes.

Fore ground process and Back ground process :-

eg: int main()
{
pf("Hello\n");
while(1);
}

→ console, when we write a.out our .exe loaded into Ram and starts executing, while it is executing I can't give any other command in the command prompt in the current terminal. Bcz my process is running in the fore ground, that is the reason, I can't communicate with the command prompt.

// when we use command a.out by default our process is running in the fore ground!!

Q → How to Run my process in the Back-ground ?

→ use command a.out &

Eg: void main()
{
pf("Hello\n");
}

→ gcc → a.out &
↓ o/p
[1] 3373 → Process ID
Hello

Let's see, what is the difference b/w Job id and Process Id and who will Assign the Job id and who will Assign the process id. ?

→ Q → still my a.out is running or not even after getting the Proper output ? → yes running in the Back ground. (in Ram)

Question ? → what is the difference b/w running the application in Background & fore ground.

Ans → logically think, when one process is running in the background, i can use one another process / run another process in fore ground.

suppose, I'm downloading one app from the internet and I will minimize it. Even though i minimize it that is running in the background. So that in fore ground we can do one more work.

→ So here also, the same thing when using a.out & → minimizing → like our phone → so running in the background.

what is the Advantage of running the Application in Background ?

Ans → In fore ground we can use, one application.

PS command :- ps command is used to list the list of processes in the current terminal → displays of.

Here ps is also one process

```
PID TTY      TIME CMD
3353 pts/0    00:00:00 bash
3449 pts/0    00:00:00 ps
```

a.out
[1]+ Done

So, all these processes are running in the background that's why I am able to do some work in fore ground. incase, if the app are running in fore ground, I can not do the any other work.

→ "All the commands are also executable files".
→ who will Assign the process ID's.
Ans] process manager. [so, process manager will not recognize the processes with the name of the process]

* process manager will Recognizes the processes with process ID.

Tell students → why we are recognizing you guys with your ID's bcz there might be, other persons also with the same name.

→ when process is created, (when my program is loaded into the Ram, for Execution) process manager will Assign one unique ID to that process, that is nothing but a process ID.

Basic Commands in Process management :-

1) ps :- it will display list of processes in current termina).

2) ps -e :- list of processes in system. (in Entire System) → can observe init with PID → 1

3) ps -l long list of processes in the current terminal.

4) fg → to move the background process to foreground.

5) size → This command display size of object file or executable file.

fg :- to move the background process to fore ground, → so last in first out → by using fg we can terminate & Again use fg the process is no more → Enter Again fg and terminates the one more process... so like this we can terminate the Back ground processes.

Confusing point with clearity.

P1.C
→ main() → cc -o a.out &
{ printf("Hello"); [1] 614
}
./a.out
Hello
Back ground Processes
[2] 615 → Hello
./a.out &
[3] 616 Hello

→ if I use fg to take back ground to fore ground the 616 will come first to fore ground (last in first out).

Intersting Question :- I don't want 616 to come out, I want 615 to come out → then what is the Solution ?

Ans] → need to type fg 2 [fg JobID) → so now this process is pushed to the fore ground from the back ground.

cessess

command for what jobs are running. → Jobs
↓ displayed as
[1] - Running ./a.out &
[2] + Running ./a.out &

Is it possible to terminate the process in back-ground itself.
Ans) → Yes → by using the kill command.

Eg:- kill -9 616 (PID)
kill -9 614 (PID)

Small conclusion:- without moving into the fore-ground, if you want to terminate the process in back-ground itself then use kill command.
→ Here -9 is nothing but a signal number, and process ID;-
this is forceful killing.

Note:- After back-ground process completion ps command automatically destroyed ? → Yes → when process is completed process manager will deletes the process-id.

Kill :- Kill is a command, to send signal to a process.

Linux-9:- Agenda :-

1) ~~Current~~ Concurrent and sequential Execution of Process.
2) process id and its parent process id.
3) some defination's
   a) Response time
   b) starvation time
   c) Turn Around time
   d) Throughput
4) states of processes.

1 Concurrent and sequential execution of process.
Already this topic done, just refer the previous topics.
if before completing P_1 process, if Another Process get CPU time → Eg: P_2. if P_2 Process not yet completed P_3 will get the CPU time.. and so on.
So, this is called Concurrent Execution of a Process. (or) Concurrent Environment

Similarly, if After completion of P_1 execution P_2 will get CPU time to execute after completion of P_2 Process P_3 will get CPU time to execute this environment is called sequential Environment (or) sequential execution of a process.

→ In multiprogramming, environment concurrent execution happens.

2] what is process ID and its parent process ID ?

Any we already know that, when ever a process is created, process manager will Assign one unique id to that process, to identify that process.

1] Each process has a process ID (PID). it is a positive integer that uniquely identifies the process on the system.

2) if we want to print pid (process id) below function we need to use.

#include<unistd.h> → it is typedefed some where & it is unsigned integer.

pid_t getpid(void);

Note :- for Every execution, Process ID's are getting changed.

Here, below is a simple program to print the process ID :-

→ getpid() will gives Process id of corresponding program.

```
#include<stdio.h>
#include<unistd.h>
void main()
{
printf("pid=%d\n", getpid());
while(1);
}
```

→ gcc → ./a.out → Pid = 44 (Process manager giving PID)

./a.out → Pid = 45

./a.out → Pid = 46

(53)

→ for same Program, we are executing multiple times but every PID's are different. All the process are terminated bcz we didn't used while(1);

→ incase if process is in while(1) loop, command prompt not came bcz Process is in foreground & still Executing. blinking :- → At the time of blinking how to taminate the process without using ^C. → Ans simply open, one more terminal, type ps -e → no matter how many terminals we opened, All the processes it will display. → so, simply use kill Command in that terminal as kill -9 53

Question :- If we kill the init, then what happened ?

Ans) Everything will get closed (terminal also get closed).

1] The linux kernel limits process ID's to being less than or equal to 32,767 defined by PID_max.

2] in Linux the PID_MAX is Available in the file /proc/sys/kernel/pid_max

1st point Explanation :- ofcourse process manager, generating Unique PID's but for that also there is limit → 32767 → (Just clude it)

2nd point → use command → vi /proc/sys/kernel/pid_max → to see max range (32768) of Process Id generation.

→ After 32768 → it will start from the beginning.
→ vi /proc/sys/kernel/pid_max → in this file, that limit is defined.
→ As a System Programmer we can't change this limit (32768).

→ To kill init process command kill -9 -1 → man init

p1.c - gcc → a.out → PID 313
a.out → PID 314 → why not 313, Already terminated so, why not it is Assigned 313 Again, process manager Assign unique ID's. It Assign only next sequential numbers, until 32767. if we reach these limit processes are not created in some cases.
Always init process process_id is starts with 1.

Process Id and Parent process Id

* Each process has a parent → the process that created it. A process can find out the process ID of its parent using the getppid() system call
ppid "means" parent process ID.

Example Code. → p1.c

```
#include<stdio.h>
#include<unistd.h>
void main()
{
printf("pid=%d ppid=%d\n", getpid(), getppid());
}
```

→ In this program we are going to print 2 informations
1st info → It's process Id.
2nd info → It's parent process ID

↓ gcc p1.c
./a.out → pid=139 ppid=8
./a.out → pid=140 ppid=8
./a.out → pid=141 ppid=8

for all these processes, whatever we are created, parent is same → with pid → 8.
that parent is Bash.

Bash :- Bash is a shell
In above code ppid is constant for all the Processes, which are running in the current terminal. (or) Command prompt.
→ In case If we open, Another terminal (or) Command prompt then ppid get change.

like → ./a.out → 146 → ppid = 15 → Bash.
./a.out → 147 → ppid = 15
./a.out → 148 → ppid = 15 → like this.

→ by this command.
PS -l :- for ps also Parent is a Bash and its ppid is 15.

What is a Bash. ?

* Bash is a shell. & shell is a command interpretor.
* shell is a user program, which is used to interact with OS.
* shell is a command language interpreter that Executes commands read from standard input device or file.
* shell is not a part of Kernel (service). it is an application.
  Eg:- shells :- sh, bash, ksh. → Types of shells.
  → we have to provide one file to shell which is having commands so that shell will Execute the those commands.
* shell is a mediator between user and O.S on the command based Environment. [on the command based environment, if you want to interact with O.S via shell we need to interact].
  bash across shell
  Bash shell
  command prompt.
Vector: ls, pwd, about → who is taking these commands as an input.
Ans → shell is taking. these commands as an input.
when we type the command, what the shell will do ?
ls
↓ shell receives the commands and interact with O.S.
and OS will create a process.(ls). and O.S will make for this
Process shell as a parent.
Bash is parent for → about, ps, ls → because that request come from bash only

Bash is one shell and there are so many types of shells are there.
don't think bash is different & shell is different. bash is one type of shell.

→ when we open command prompt, who is waiting for the input ?
Ans → shell → which shell → Bash shell.

Interesting discussion :- Bash is also one process so, bash is also having parent. → ps -f. (58:30)

PID PPID — bash → so bash parent id 7.
8 7

How to know whose process id the 7 is. → ps -el
then we come to know that 7 is init process.
for init also having parent → with PID → 0 → i.e Process manager (or) scheduler.

ps -el → Entire terminal processes. here 'l' means long list.

$vector ÷ → who is displaying command prompt like this?
Ans) Bash shell

Whenever we open any terminal by default Bash shell will run.
Now very interesting → type command sh (sh is also one type of shell).
then interface is changed in the command prompt, because now bash shell is not waiting for the input sh shell is waiting for the input.
After → sh command → a.out → pid=152, Ppid 151 (just write code to print pid, ppid).
→ $ now use ps

PID
151 → sh

Very Very Very Very interesting Point:- Who is the parent for sh.?
Ans → Bash → How? very very simple from Bash only request come for sh...
* → Bash command prompt is different and sh command prompt is different, only $ symbol.
$ bash → now sh is taking this input → hit Enter we will get.
bash command prompt Again. → Now use ps command → Enter.

tty1 → means terminal 1
tty2 → means terminal 2
tty3 → means terminal 3.

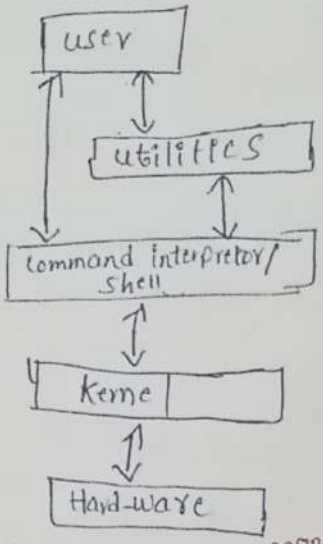
PID	TTY	TIME	CMD
8	tty1	00:00:00	bash
151	tty1	00:00:00	sh
155	tty1	00:00:00	bash
163	tty1	00:00:00	ps

now in command prompt which is waiting for i/p → it is 8 or 155.
Ans → 155
Now → If I write a.out I didn't get 8 as ppid I will get 155 → as ppid (simply write code which is printing pid & ppid).
Now try to understand what happening is. → By default when I open this application (terminal) command prompt will come and bash will started executing (by default)
my doubt → is Command Prompt and Bash same.? → No command prompt is displayed by bash shell... similarly sh shell. etc.
Give sh → command Again → sh command prompt will opened, and whatever the command giving further, for that command the parent is sh. → which sh → ? bc3 After typing ps so many sh will come because we use sh many times. So Ans is last sh. → write a.out (simp code same pid ppid display)
we can observe just before pid of sh come as ppid of the process.
if I use Exit command from sh command prompt → so that sh will terminate.
Interesting. Exit → sh terminate bash command prompt open → Exit sh → command open → Exit
bash open → Exit... keep on going how many times we opened it.

why these many shells → Bash, sh, ksh. → Bash and sh are different Programs but work is same. (like browser is same why Chrome same here also)
→ In bash some short cuts will work Eg control-l means it will clear the screen but in sh it will not work.
* so bash is user friendly shell compared to the sh. and also in bash shell we if we type tab so remaining letters Automatically comes. but in sh shell can't work like that → o/a → give tab

There are so many types of shells are present in our system, to know, how many shells are present in our system, simply use Command.
→ cat /etc/shells → hit Enter → All the listed shells work is same.

what is shell :-



Terms related to process management :-

(1) • Response time :- The time gap between the process loaded into the Ram, and its first instruction executed by the CPU.

Eg :- Processes

→ Consider when process is loaded immediately, that process not going to be executed. (dont expect like this)

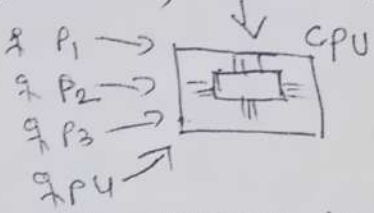
is this P4 started executing immediately when it is created or loaded.
Ans → NO → Because process manager, will give ~~time slot~~ cpu time slot for Every process.

The time gap blw the process is created and the first instruction of the process executed by CPU that is called as Response time.

Response time is more is good Environment or less is Good Environment?
Ans) → less is good Environment. → simple Answer.

Starvation Time :- A process in its life cycle how much starved without executing by CPU, this time is starvation time.

Eg: Now consider, Concurrent Execution



when process manager given cpu time to P1, then remaining processes are starvated.
Similarly, when cpu time is given to P2 → P1, P3 and P4 are starvated.
Similarly, when cpu time given to P3 → P1, P2, and P4 is starvated.

1:17 starvation meaning → stopping → English meaning (stoping without executing).
Process in its life cycle → life cycle means process creation to process completion → how much of time it is waiting for the CPU time slot to get → that time is called starvation time.

more starvation time is Good Environment or less starvation time good Environment?

Ans) if the process is starvated less is a good Environment.

(3) Turn-Around Time :- it is a time gap between process loaded into the Ram and its Execution Completed.

in Turn around time starvation time is present, & in starvation time, response time is also present.

Turn Around time more is good Environment (or) Less is good Environment.
(or)
my process is completed faster is good (or) slower is good. ?
faster is good "means" Turn-Around time is less is a good Environment.

4) through put :- the number of processes that are completed per unit of time is called through put.

Q through put is more is good (or) less is good (or) more no of Processes Completes faster within a time is good (or) less no of Processes Completes with a time is good. ?

Ans) more no of processes completed within a time is good.

Through Put is more is a good Environment.

3) Delay state process :- If the process intentionally, goes to delay, then the process is treated as delayed state process.

Eg → sleep() → Whenever we use sleep() in our program then our process is goes to delayed state process. (We categorized it as delayed state process).

4) Suspend state :- if the process is suspended because of some signal, then that is a suspend state.

(Simply All Above 4 states are waiting for something).

We will Analyse Above 4 states programatically also!!

→ open terminal vi header.h

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<string.h>
#include<sys/types.h>
```

→ At time of Ending linux module, we need to use 20 to 25 Header files. we need to use.

→ In Kite Program we need more header files, so that why sir habituating the concept now onwards only.

vi p1.c

```
#include "header.h"
void main()
{
if("Hello Pid = %d\n", getpid());
while(1);
}
```

According to Programmer, while(1) Means infinite loop.

gcc p1.c -o p1
↓ creating own executable with p1. instead of .exe

Q → How many states, the above Process is changing ?

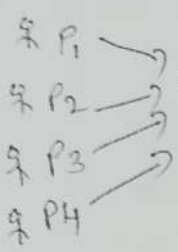
→ gcc → a.out → Ram

→ Whenever our program is loaded into Ram, it is not executing immediately so it is in Ready state. → bcz already so many processes running so, this process needs cpu time, when our process gets, cpu time then it is in running state. → don't think it will Executes complete program, how many instructions that cpu can able to execute, that many instructions only executed in this slot, & rm Assigns cpu to Another slot (Bcz it is concurrent Execution). → one more important point is our program is in while(1) loop. if cpu executing only our process then other processes can not get the cpu time slot.

while(1) :- Even though it is in while "1" don't think that, it is always in Running state → if it is like that then other processes can not get cpu time. → it is possible only in sequential execution (cpu execute 1 process completely then only executes other process that is also complete execution it is called sequential Execution).

Linux-10 Process management Sub-topics :-

4) states of processes :- Consider, multiprogramming Environment (or) Concurrent Environment



→ We know that at a time only one processes can execute, lets say P1 is executing right now, so that P1 is treated as running state :-

And Remaining processes are in other state don't think they are in starvated state Because starvation is not a state.

* If the processes is not executing by the CPU then the process may present in the below states.

1) Ready state
2) wait state
3) Delay state
4) suspend state

* if Process Executing by cpu then we need to say, it is in Running state.

→ if the Process is purely waiting to get cpu time then that Process is Treated As Ready state process. [if the process is not waiting for Anything Else only waiting for cpu time to get i.e Ready state Process]

* In Above Diagram, we can treated as P2 in Ready state process

* if the process is purely waiting for cpu time to get the process manager treats (or) categorised the process as Ready state Process.

2) wait state process :- If the Process is waiting for external Event to finish.

Eg :- scanf → when we use scanf in our program then it is a blocking function, it is waiting for user input [means it is waiting for External input, then the process is treated as wait state Process.

our Program when in Running state, if other process get cpu time slot so our process again goes to Ready state. from the Ready state, once Again time slot is given by the Rm to our Process, then Again Running state our process will & vice versa. it is happening because of while(1). In case if no while(1) our process might have completed in first slot itself.

And our process is always going to two states only Ready → Running, Ready - Running, Ready Running, Ready Running - - - - - ..

Note :- our process is not going into Delay state because we are not using any sleep function in our program.

question ? In Real time, where we observe that only some instructions are executed in 1 slot and remaining statements are at pending for next slot ? → In debugging tools only we can observe → practically not possible in this general purpose Environment.

Do simple Task → open one terminal & write one code (Print Pid in that code) and Again open one more terminal → and give the command ps -e

PID		
tty1	00:00:35	a.out → now we are in terminal 2 tty2
tty1	00:00:00	init → meaning from the last 35 secs it is running in tty1.
tty2	00:00:00	bash.
tty2	00:00:00	init [But in that 35 secs only, it is going to Ready state, Ready to run, Ready run, Ready Run like this].

↑ But overall we can say from last 35 secs it is running. (present)

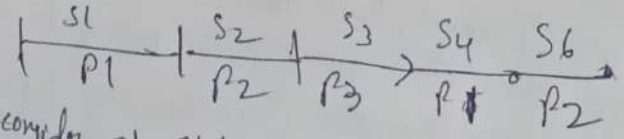
ps -el.

"ps Means Process list e "means" Entire terminals, how many terminals are there, all the terminals.

"l" Means long list.

* in concurrent process → P1, P2, P3, P4

cpu time slots.



consider at Slot 7 while Executing P3 → & P3 having very less instructions than P4 until is the slot 7. Also (if not Completely somewhat).

Now lets talk about wait state programatically :=

eg :-
```
#include "header.h"
void main()
{ int num;
  Pf("Hello Pid = %d\n", getpid());
  Pf("Enter the number.....\n");
  sf("%d", &num);
  Pf("num = %d\n", num);
  while(1);
}
```

Student doubt :- How the CPU knows that process is completed before the time slot.?

CPU doesn't knows Anything, CPU only knows that in a program counter, whatever the Adress is there, it will executes that instruction. All the things are managed by process manager only.

Note :- CPU don't know Anything → CPU only knows fetching, decoding, Executing, fetching decoding Executing, fetching decoding executing that's it. Even CPU doesn't know which process instruction it is Executing...

note :- → we are working in first P4. then P3 like this. (All processes are having the same priority).
In General Purpose Environment there is no priority of process to execute
In real time Environment, there is a concept of priority.

Now observe the above code, when I load the program into the Ram first it will go to ready state then running state & from running state it will go to wait state because of the scanf. (waiting for i/p).

Can you tell me, when it come out from the wait state ?

Ans) when user supply's the input.

Again go to Ready state → Run state → Ready → Run ----> beg of while(1)

Very interesting Point :- compile the above code in terminal 1 cc -> .\a.out.

o/p -> Hello Pid = 65
Enter the number..... (now without Entering value, open the terminal 2)

waiting for External Event to finish

→ After Entering the value, Again, open terminal 2 and type ps -e

```
PID  TTY  TIME      CMD
65   tty1 00:00:00  a.out
```

↓ ps -e

time → because not yet Enter value it is in wait state

so now we can observe time is increasing.

(beg of while(1);) it is not yet terminated, so time increasing.

↳ we can kill the process -> in t2 -> kill -9 65 -> Enter.

*Delayed state :- Example code :-

```
#include "header.h"
void main()
{
  int num;
  Printf ("Hello pid = %d\n", getpid());
    sleep(10);
  Printf ("After sleep---\n");
  while(1);
}
```

→ gcc → .la.out → pid = 100

open terminal to
ps -e
↓
you may observe now
a.out time is still 00 00 00
Again give ps -e
now you may observe, a.out time is increasing because it may get out from that ~~stopping~~ delayed state.
now time is keep on increasing because of that while(1);

⇒ How many states the above program is going :-

R/ready → run → ~~wait~~ delay → ready → run — ready run.

Can we predict it, when it will come out from the delayed state?
Yes, we can predict, because we are only giving 10 secs, so we can predict after 10 seconds it will come out.

Can we predict when it will come out from wait state?
Ans) we can't → bcz purely depends on, when the user is going to supply the input.

Note :- When any process coming out from the wait state (or) delayed state it is not directly going to the running state, bcz in that moment other processes might have running, that's why it will go to Ready state immediately after it is coming from wait state or delayed state.

Tricky → ## Suspend state :-

By sending one signal, we can suspend the process

```
#include "header.h"
void main()
{
Printf ("Hello pid = %d\n", getpid());
  while(1);
}
```

→ gcc → ./a.out → pid = 127

→ one open one more terminal, & give command ps -e

pid	time
127	00:00:20 s

→ from last 20 secs our process is executing.
→ now → kill -19 127 (19 is a signal number for suspend (or) stopped.)

Now, After giving the command, → kill -19 127. → Again → ps -e.
now we can observe stiu our process is showing running time of 20 secs only
Because our processes is suspended or but not terminated.
if our process might have terminated, ps command not displays our process
info → tty! time 00:00:00 about → like this we can not get.

kill -9 → for termination of process
kill -19 → for ~~term~~ suspend the process
kill -18 → for resuming the process.

So, it is so clear that kill means not used for only termination it is used to send a signal to a process (kill is a command to send a signal to a process). (to interupt the processes). → in signals concept, we can discuss more.

Note :- we are suspended the process above, → kill -19 → That's why it is still showing time as 20 secs → if we Resume this process it will start Running Again from 20 s → how to Resume the process? → kill -18
now use ps -e After using kill -18 → so, now we can observe that about time started increasing. (Bcz that suspended process is Resumed).
Again suspend → kill -19 127 → ps -e → about time 50 sec every ps -e
& resume it → kill -18 127.

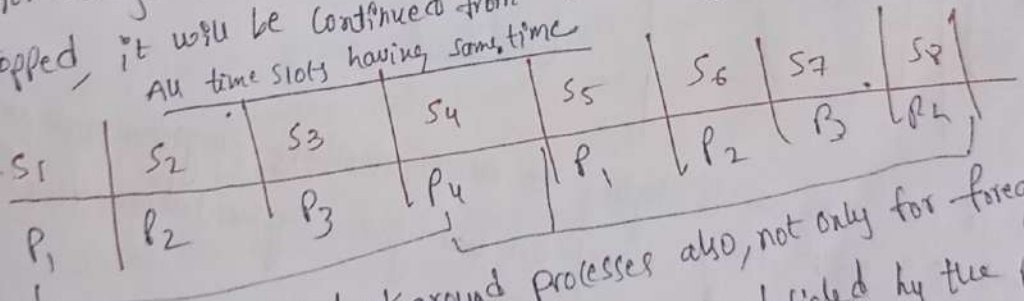
Note :- if the process is comes out of the wait state, so that time is once again Reset to zero. ↓ suppose → At the time of user i/p → 00:00:10. → but After External Event, means, whenever user gives input → Again the time is Reset to zero.

final Conclusion :- if the process is not executing by the CPU, then process will go Any one of state. (Ready state, wait state, Delay state, Suspend state).

Note :- when our process is suspended, it is Automatically goes into the Back ground.

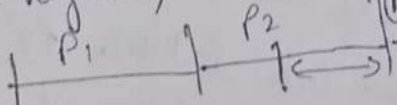
Note :- for delayed state (sleep() used), the time is freezed, & from where it is stopped, it will be continued from there only. (But wait state, After user give i/p set to 0).

Note :-



These time slots are for background processes also, not only for foreground Processes, & already know's very well, the time slot is decided by the process manager.

Solution



if P2 Process completed within half time slot, is remaining Half time slot, CPU is in Idle state → no P3 will get that P2's half time slot.

* if the CPU is in idle state Performance gets decreases.

Note: In delay state also, don't think that, CPU is not working at the delayed time, bcz CPU never stops until we power off the system.

Very interesting point: when we open the terminal, it is waiting for the input. (or) it is waiting for the commands, so we can say that bash is in wait state (external Event) → ./a.out. → After giving commands to bash, (Bash is waiting for user input).

means ./a.out → hit enter, Again, my bash is in waiting state only, bcz for next commands i/p (or) user input. (So most of the times, bash is in waiting state only.

Note: That's why when we use ps → command bash time is always 00:00:00 because it is in waiting state immediately after taking the input for next input.

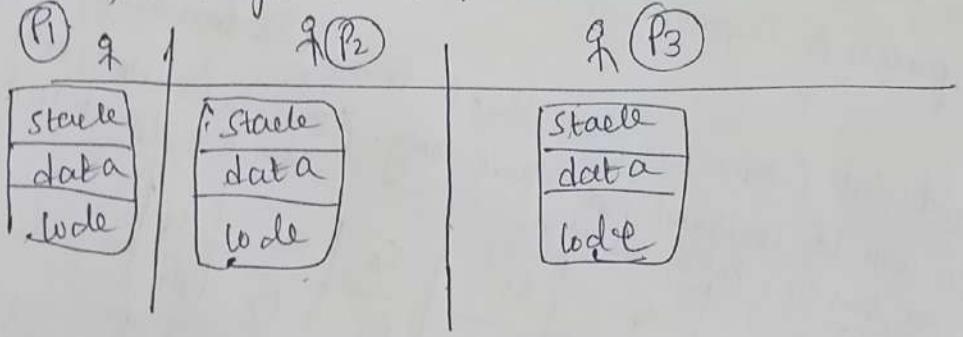
Note: if the process is in waiting state, process manager will not give time slot to that process.

→ wait state means if a process is waiting for External Event to finish like a scanf ie called wait-state (or) process goes to wait state.

Linux-11 :- Agenda :- Dt 07/03/2025 → 7:42 Am

1) PCB (Process control block)
2) Context Switch
3) System calls v/s library functions.

* Sir is telling that, when P1, P2, P3 Processes are loaded into Ram, each process is having its own → stack, heap, data and text section → That's fine But the question is he is saying that → for every Process 8mb stack will be Allocated, I thought that complete stack size in a Ram is 8mb.



Note: if one Program is trying to Access, Another's Program memory then what OS will do? → Ans) OS will give segmentation fault it is un-Authorized memory Access.

So Every process having its own memory limits. like → P1 having its own stack, data, and code section. P2 also having its own memory limits → ie stack, data, code. and similarly P3 also.

Question → P1 process instructions are present where ? (or) uploades are present where?

Ans) → Code section of P1, similarly P2 process instructions are present in Code section of P2 → similarly P3 also same.

P1 | P2 | P3

→ All the Process P1 P2 & P3 having main().

We know that very well → When single CPU is there & multiple processes needs that single CPU → so, Processess manage need to manage.

2nd) known Point only when P1 → Process get CPU time slot by PM, then P1's ~~all~~ all instructions are not going to execute. CPU only Executes how many instructions it is capable of Execute in that Particular time slot. remaining instructions are going to Execute in the next slot period.

Note :-

P1	P2	P3
having 100 instructions → Executed 25 1st slot	→ 100 executed 35 2nd slot	→ 100 Executed 50 instructions in 3rd slot.

Question → why slot 1 of P1 → 25, slot 2 of P2 35, slot 3 of P3 → 50

why not Equal instructions → simple Answer it depends on instruction size also :- may be P1 instructions are more lengthy that's why 25 instructions are executed → so clear.

→ Now, After Executing P1, P2, P3 Again CPU has to Execute P1 from 26th instruction onwards → question → how CPU knows, that it has to Execute from 26th instruction onwards. [there is some internal Mechanism] → PCB (Process Control Blocks)

↑ what is the mechanism behind this Ans → PCB. (18:00)

⇒ PCB Process Control Block. (PCB) :-

our processes are present in user's space (offcourse in Ram)

Imp Note :- whenever a processes is created, what the Process manager will do means → Process Manager will Create one structure for that process in Kernel space. that structure is called PCB. for Every process, separate structure is Created by Process manager. → like PCB of P1, PCB of P2, PCB of P3 ... ∞

what that structure Contains ? → That structure Contains, information About that process.

where the PCB's are Created ? → In a Kernel space.

Now what is that PCB Contains ?

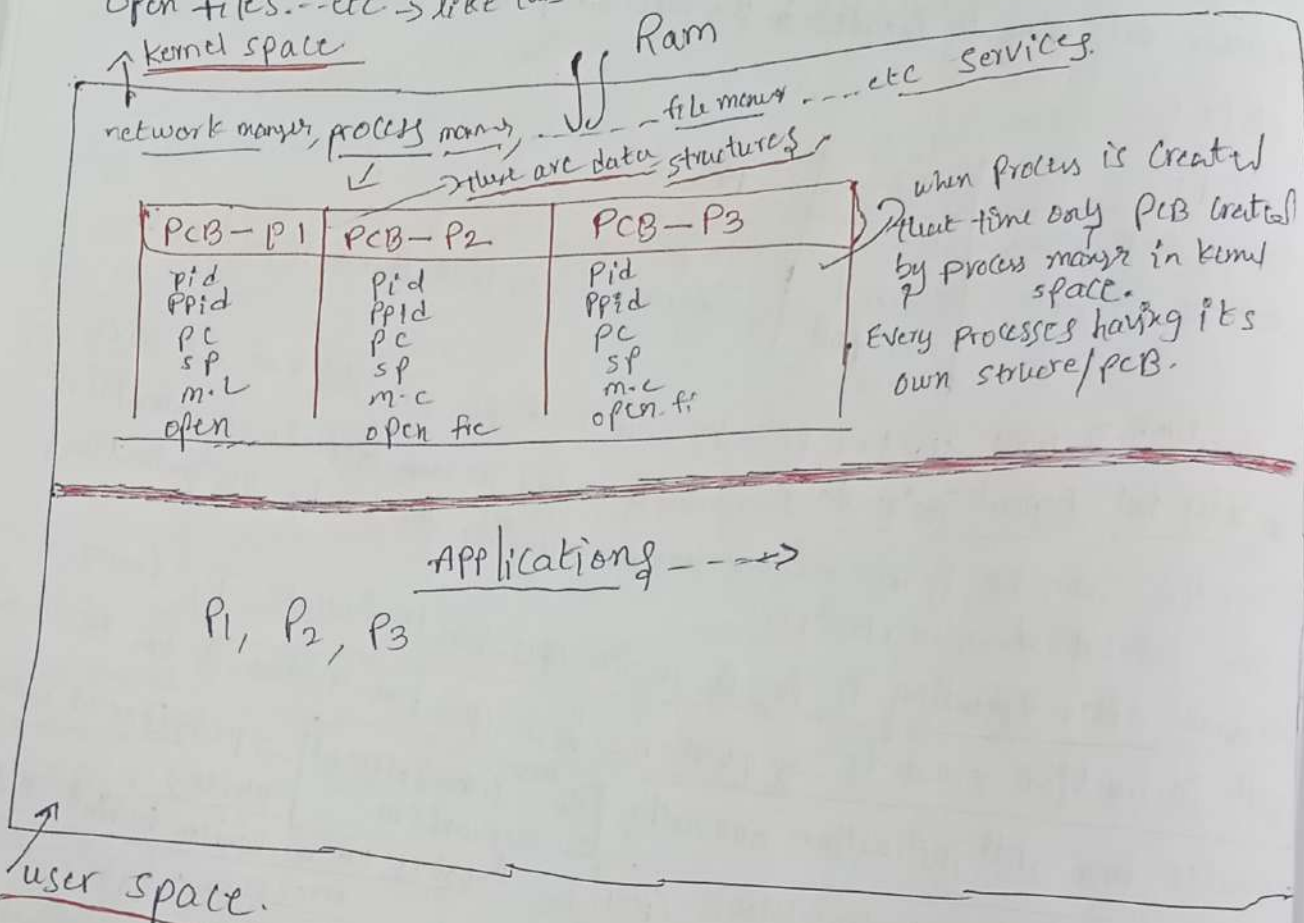
→ like in our student data base structure, it contains student name, Marks Age, roll no --etc will be there.

→ when process is created the Process Manager will creates one structr. that structure is technically called as PCB:

That PCB Contains information of process.

what information it Contains ?

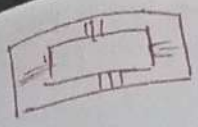
Any PID, PPID, program counter, stack-pointer, memory limits, Open files.--etc → like this so many informations will be there.



PCB-P1	PCB-P2	PCB-P3
Pid	Pid	Prd
Ppid	Ppid	Ppid
PC	PC	PC
SP	.	SP
m.L	.	m.L
open-files	.	

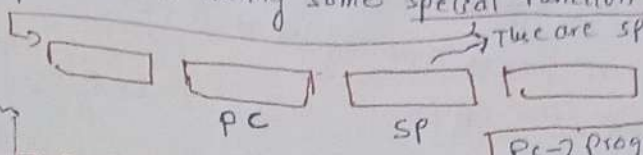
→ this is like a linked list.
one structur/PCB one struct/pcb
Another struct → Another struct
→ Another structu → Another structu.

In our discussion 3 processes are there. these processes are present in the user space, but the PCBs are present in Kernel space.

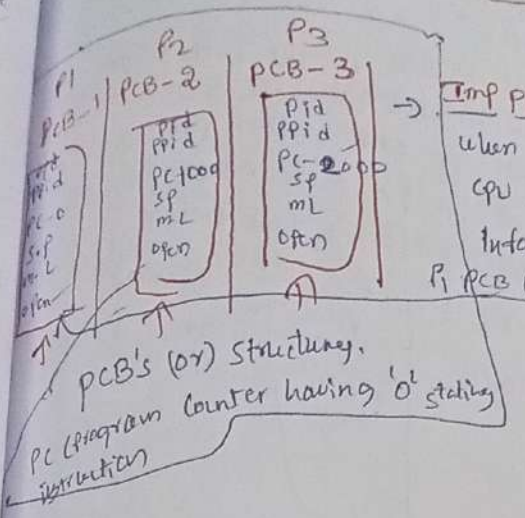


CPU.

This processor is having some special function registers,



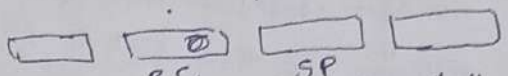
PC → Program Counter.
SP → stack pointer.



Imp Point :-

When Processes manager decided that, it need to give the CPU time to P1 process, then process manger loads the information/PCB of P1 into the CPU.

P1 PCB load into CPU →

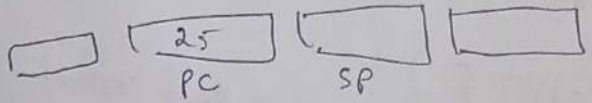


Now P1 process program Counter is holding "0". from 0th instruction it will start, or incase PC having 1000, start from 1000.

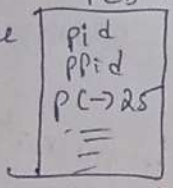
Take P2 process code section → 1000 Adress

Take P3 process code section → 2000 Adress.

Now Consider, when P1 is in execution means it is getting time slot now lets say 25 instructions are executed in that time slot [we know that how instructions are possible to execute in that time slot that many] when 25 instructions are executed, time slice expired & timer interrupt came to process manager.



So now its time to give the time slot to P2 process (next slot is P2 process) so now P2 process PCB should be loaded into CPU registers by Process manager. so that it knows that CPU instructions are executed from 1000. [But before loading the P2 process PCB, -P1 process PCB. → which is present in a CPU registers that are saved in kernel space



→ why need to save like this? because whenever Again P1 gets CPU time slot. So that's why they are saved (or) unloaded to respective PCB.

What is the need of unloading from CPU Registers to PCB-1 ?

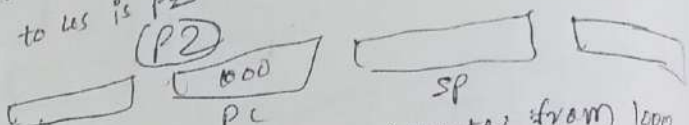
Ans) Already explained above Again explaining → next time when the time slot is given to P1 so it need to execute from 25... so to know that information, it will preserve the latest last information into the P1 process PCB.

→ How to know, how many processes are completed?

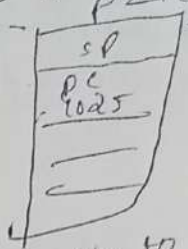
Ans) Program Counter holds the next instruction Adress to execute, so when time interrupts comes, CPU stops executing & PC SP memory limits Pid PPid etc will be stored into the respected PCB → we know Already why it is like that.

info → 25th instruction it's suppose to Execute but interrupt Came to the process manager;

so where it is stopped that information, is saved into the respected PCB. so next process according to us is P_2. so P_2 process PCB is loaded now into CPU. →



Here program counter holds 1000. so lets say it will execute from 1000, 1001, 1002, 1003 ... upto 1025 for suppose. At the time of CPU executing 1025 instruction again timer interupt came, then process manager decided that now time is completed for P_2 and i have to assign CPU time slot to P_3 (which is in Ready state). but before ~~P_3 is 1001~~ P_3's PCB loaded into CPU, P_2's info (or) updated info should be loaded into Kernel spale



so that whenever P_2 process, getting again time slot, run next instructions are going to execute from, 1025 location. (so that's why P_2 updated info is unloaded from CPU to Kernal spale (to PCB-2).

This loading and un-loading the PCB's from the CPU registers and to the CPU registers that is called Context switching :-

Q→ when Context switching is happen ? ?

when the CPU is switched b/w the two processes, then only context switching happens. [when time slice of one process expired (or) when timer interupt comes].

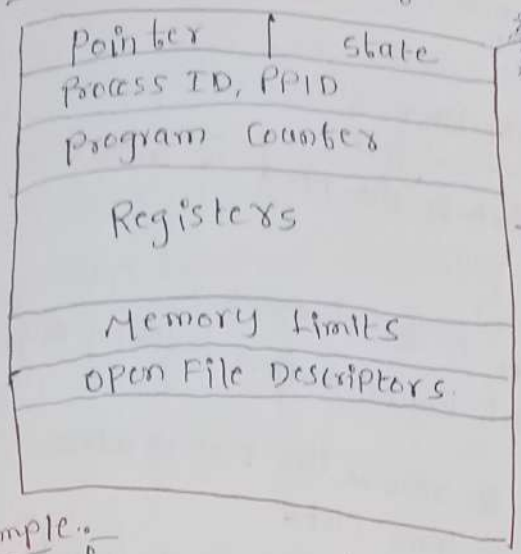
who will do loading to CPU & unloading from CPU to respective PCB ? → Process manager.

PC 1000 PC 2000 → These are the memory Adreses, where the upcodes are there. so these are the location Adresses.

PCB (Data structure) contains following information.

1) Processess state
2) process ID, its Parent ID
3) program Counter
4) CPU Registers
5) CPU shedding information
6) memory managment information
7) List of open file descriptors
8) Accounting information
9) I/O status information.

Process control Block :-

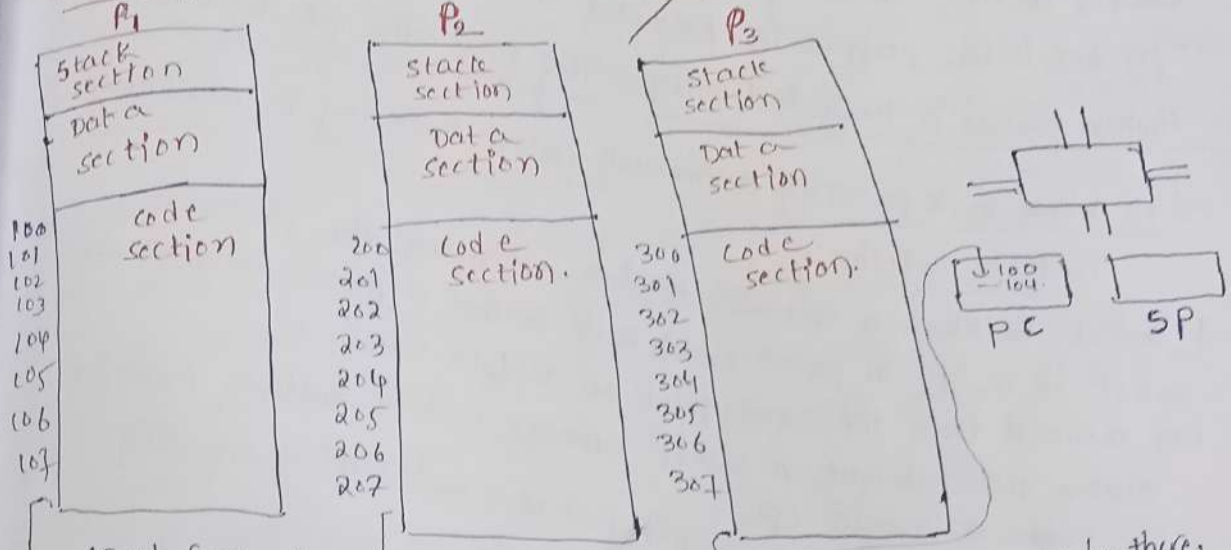


* This PCB is a data structure.
* For every process there is a separate PCB will be there.

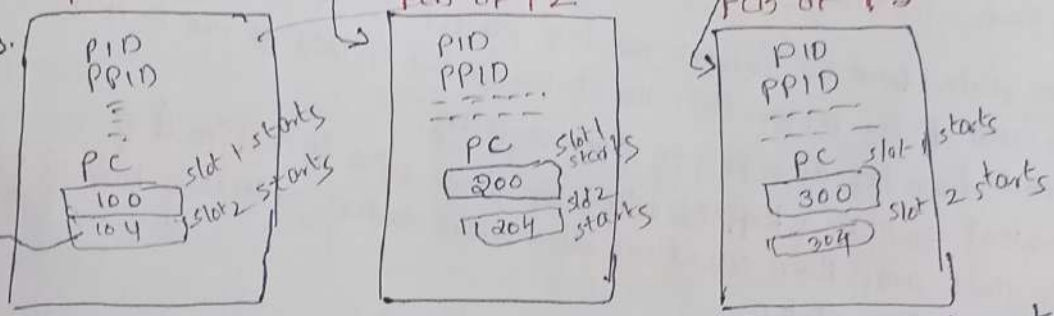
In below diagram what is 100, 101, 102, 103, 104 ... .
Those are the locations, where upcoming are present.

(Who will decide the time slice
Ans → process manager will decides the time slice.

Example :-



Apart from above diagram for every process separate PCB will be there.



Now, for suppose process manager decided to give the CPU time to P1 process → what is a meaning → then P1 process PCB should be loaded into CPU Registers., now PC holds 100 now → so now 100, 101, 102, 103 executed, at 104 time slot is Expired → so upto 103 only it is successfully Completed. so now that current information, preserved in respective PCB.
→ NOW P2 process PCB is loaded into CPU means 200 is loaded → so, 200, 201, 202, 203 → when 203 is executing PC holds 204. when PC holds 204 and time slot Expired now Pm decides to give CPU time to P3 → ie 300. before 300 (P3) loaded into CPU P2 current info in CPU (204, PID ... etc into) stored in respective PCB. (or) All other informations are unloaded into respective PCB.

so loading and un-loading of PCB is known as context switching:-

what is the need of context switching ?

In multi-processing Environment, what is the need of Context Switching ?

IF context switching mechanism is not there, for 2nd CPU time slot also PI going to start Execution from beginning only.

"Simply context switching is helps to resume the process, where it has been stopped in previous time slot.

"when process is terminated, corresponding PCB also get destroyed"

-> "To Achieve multi-processing Environment, context switching is mandatory"

Context switch :-

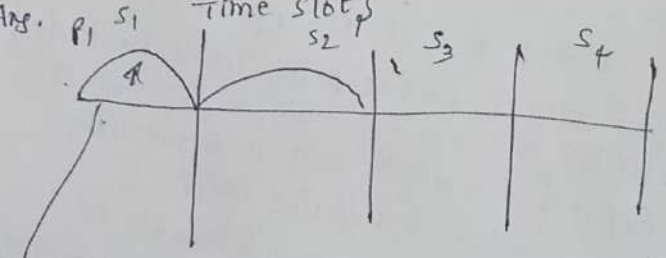
A context switch is a mechanism to store and restore the state or context of a CPU in process control block so that a process execution can be resumed from the same point at a later time.

* using this technique, a context switcher enables multiple processes to share a single CPU

* context switching is an essential part of a multi-tasking operating system features.

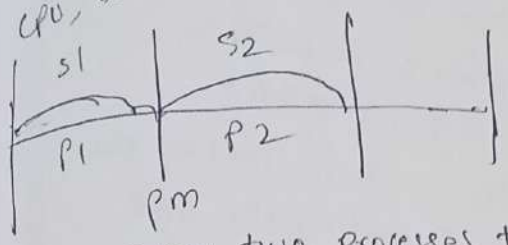
* the state from the current running process is stored into the respected PCB. After this, the state for the process to run next is loaded from its own PCB to CPU registers.

-> is context switching happens only when time slice is expired ?
(or) is there any other situations also where context switching happens ?

Ans. 

-> is there any chance a process is going to leave the CPU before time slice expired ? Yes -> 1) A process may be suspended
2) A process may go to wait status
3) A process may be delayed.
4) or process completed also

Normally, whenever time slice is expired, then only context switching happens. (or)
whenever time slice is expired, then only the loading and un-loading the PCB's takes place but there may be a chance, before time slice expires also, the context switch happens. eg if the process goes to any of the state (Delay, wait, suspended). so in this case also, there is a Chance of Context switching.. because the running process left the CPU, before time slice has been completed.



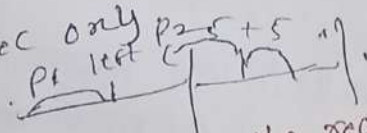
In blw Every two processes, the process manager will Executes because for loading and unloading the PCB's. (BC3 Process manage also one Program, one Process) for Context switching.

→

Question :- Consider 3 Processes are running and Each Process CPU time slot is 10 secs, what is P1 left early by 5 secs only.

Any) remaing 5 secs will be utile by P2, but don't think that P2 will get P1's time slot 5 secs and its own time slice to which is 15 mins no. wrong. P2 Also get offcurs P1's 5 sec

+5 sec only



* what might be the reasons that P1 left the CPU Early?

Any) may be it might have gone to wait, delay, suspended states.

2nd → reason may be P1 Process Completed within 5 secs this is also possible.

* System calls v/s library functions

library functions	system calls
1) supported by compilers.	1) supported by operating system.
2) library functions are also functions.	2) systems calls are also functions
3) Another name is Application Programming interface (API)	3) Another name is System call interface (SCI)
Eg:- Here we write programs using libraries i.e called API	Eg:- if we use system calls, for writing programs i.e (SCI)

library function's

3) writing the programming with API's is called application programming.

4) They are slower but process calling library fun's execute faster.

5) library functions are programmer friendly and they are used for doing specific tasks.
Eg: to read a character fgetc().
to read a line fgets().

6) writing the programs using library functions is easy. because for every task, there is a separate function will be there.

7) library functions executes in a user space.

system call's

3) writing the programming with SC's is called as system programming.

4) system calls are faster but process calling system calls execute slower.

5) system calls are os friendly and generic in nature.
for eg: read().
Generic means" for reading the character & reading the line also, we have to use same system call i.e read();

6) writing the programs using system calls is difficult as compared to library fun's. Bcz for multiple tasks only one function will be there.

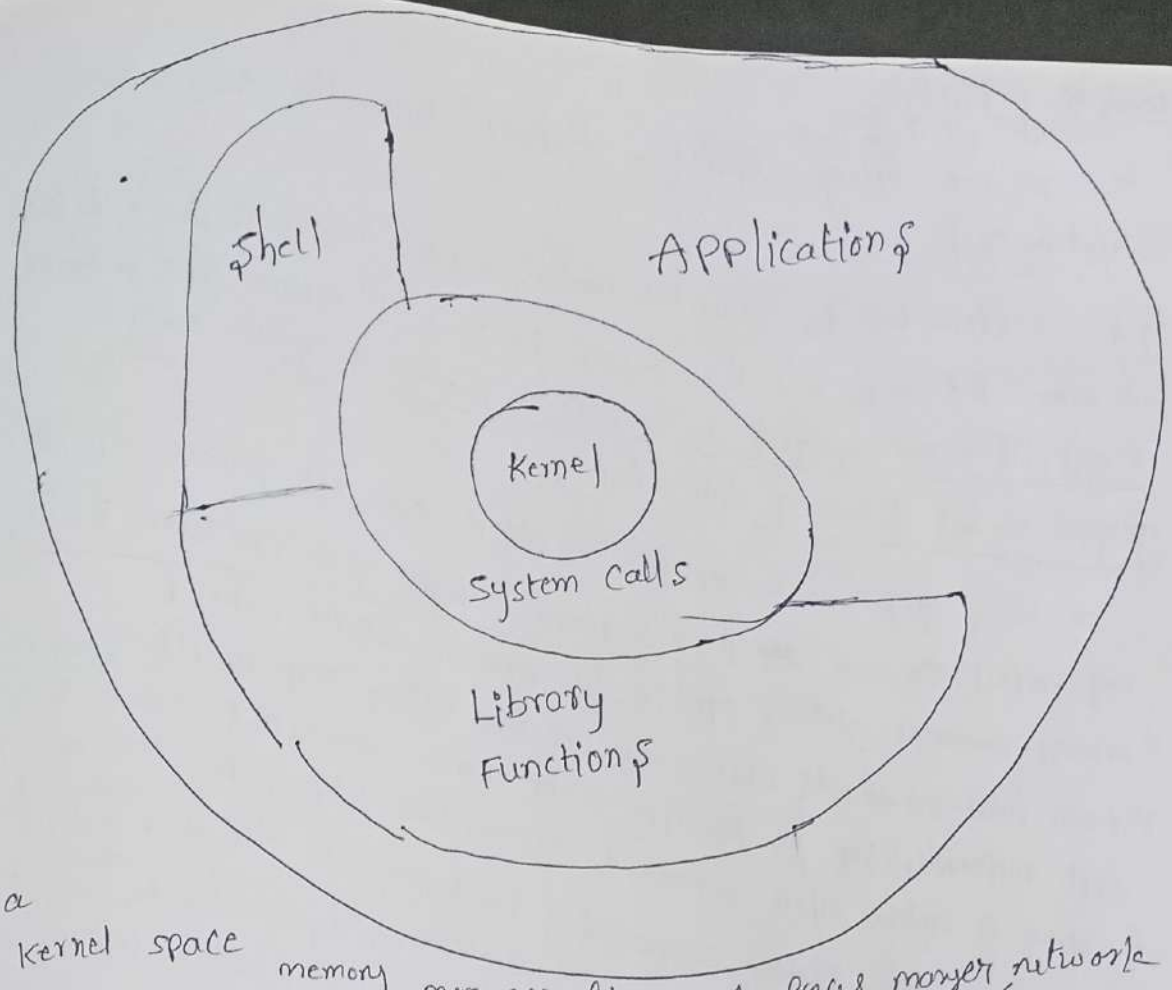
8) system calls are executed in a kernel space.

Imp point: if any process is calling the system call, the process needs to change its state from user mode to the kernel mode.
the process itself is goes to the kernel space, it needs to execute there and it needs to come-back.

Q) Till now we might have written a programs using sys library functions → Then what is the Necessity to use system calls ?

Ans) our library functions can not support to communicate with all the managers

→ (sys mins)



In a Kernel space memory manager, file manager, process manager, network manager etc All the manager are there.

Also operating system designers, designd some functions to communicate with these managers. (those functions are system calls).

But our c-library may not support that, 'means' compilers does not have these many functions to communicate with the managers.

question?
why we need to go for a system programming, why not application programming using

Ans) our gcc compiler will not support to communicate with all the managers which are present in the kernel space.

if our compiler is not supporting, we need to directly interact with the system calls.

How to identify the function is system call or library function?

use command man man

3 "means" library fun → we can observe this after typing man man.

2 "means" system call

↳ C, 2, --- 9 categories are available (man man for).

" [1] " for commands

Give command

→ man strlen. → o/p strlen(3)

if compiler need to communicate with the O.S, Again it is also communicate via ~~OS~~ system calls only.

Interesting Facts:-

Eg:- we are using fopen(). fopen() internally calls open system call

Eg 2:- suppose we are using the malloc, malloc internally calls brk().

→ man ls → it will display → LS(1) → '1' means shell command.

* getpid → man getpid → GETPID (2) → which is system call

getpid is not library function.

so, using getpid → we are asking please give me process ID of my process, so that process manager will give PID from PCB.

* process manager giving PID in the form of return value

* from now on-words whenever we are using any function, our first responsibility is to use man command to check that function is either shell command, system call, library fun etc.

* we can write system programming by using ctt also. other languages also categories...

* but here we are going to write system programming by using C.

BC2 most of the linux operating systems are designed by using C only.

→ All the managers are designed using 'C' and all interrupt concepts and all other things.

so simply in this module we are going to communicate with the managers with the help of os supported functions.

→ (3) → library function.

* System()

Prototype:- int System (const char* command);

* System function is used to execute a shell command.

for Eg:- for strlen() → we need to pass string base Adress, so that starting from Adress until it founds null character it's calculates the string length.

* for system call also we have to pass string:-

Example code on system() :-

```
#include"header.h"
void main()
{
  printf("Hello---\n");
  system("ls");
  system("pwd");
  system("cal");
  printf("Hai----\n");
}
```

→ And already we know that string name represents Base Adress.

→ use one code →

```
main() p1.c
{ pf("vishnu");
  sleep(10);
  pf("vector\n");
}
```

→ gcc p1.c -o vishnu

now left side code After system("ls");
we → system("./p1"); is null);
So o/p → After listing files.
→ vishnu → gets printed.
→ vector
→ calendar display

waits 10 secs After Printing vishnu

./a.out

o/p

Hello
p1.c a.out p2.c head.h → // ls command is Executed.
home/guru/linu_8a/pm → pwd command is Executed

April 2025

sun	mon	tue	wed	thr	fri	sat
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

→ cal Command is Executed.

$cal → also display Calender.

```
main()
{ system("clear");
}
```

→ it will clear the screen.

Note:- simple note Any shell Command if we want to execute, we can use → system(); in our code.

Note point:- if we are calling the system() in our program, so in our process internally 2 more processes are created.

① → sh ② → whatever the command that we pass via system("ls")
ls. → created.

* if we give wrong command in system("abc") → it will generate Error.
as → abc command not found. → ("sh is displaying this Error).

system() :- if you want to execute the shell commands, through your program then we need to use system() .

Linux-12:- main topic process management:- 08/03/2025 -> 5:15 Am

Agenda :-

system()
perror()
fork() -> very important system call that's why need to discuss 2 to 3 days

-> system() :- it is a library function. it is compiler supported function.
-> the Argument in system function is string base Adress, that string should contains commands.

Basically, system() is used to execute the shell commands by using by writing the code (not on command line prompt).

compile time i/p

```
system ("ls");
```

Run-time i/p

```
char s[10];
scanf("%s", s);
system(s);
```

system() function :-

* The system() function allows the calling program to execute a shell command.

eg:- int system (const char* command); -> prototype.

* system() executes a command specified in the brackets by calling /bin/sh -c command, returns after the command has been completed.

eg:- system ("ls -l") -> now this command is executed in sh shell.

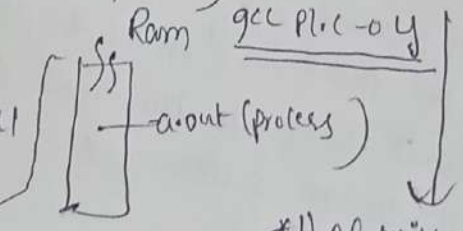
and upon failure it returns -1.

Example code :-

```
#include<stdio.h>
int main()
{
printf("hello\n");
system("ls");
printf("hi");
}
```

-> who is Parent for this process ?
Ans) obviously Bash is parent for this process.

* in a.out process 3 instructions are there they are:- printf("hello\n");
system("ls");
printf("hi");



After gcc -> we will get .exe -> a.out.

* ll pf will print the data on the screen.

2) * when we call system("ls") -> internally two more processes are created *

→ who is calling system() → C-programming point of view main is calling the system().

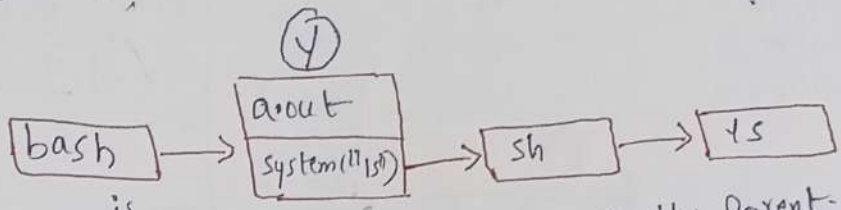
→ In operating system point of view, Process y is calling the system().

→ whenever the process "y" is calling the system() → internally 2 processes are created, 1] sh 2] whatever the command we supplied in the system("ls");

↳ 1) sh 2) ls.

Again eg. :-

```
#include<stdio.h>
int main()
{ printf("Hello\n");
  system("ls");
  printf("Hii");
}
```



→ who is parent for sh process → y is the parent process.

→ who is parent for "ls" process → sh is parent process for ls.

→ who is the parent for this a.out process → Bash.

now, tell me how many processes involved here?

1] → bash → a.out → sh → ls.

→ system();

"process" means A program which is loaded into the Ram for execution.

Very imp question: when the "hii" statement printed in the Above program?

whenever ls command execution is completed then only hii statement is printed & following statements will execute.

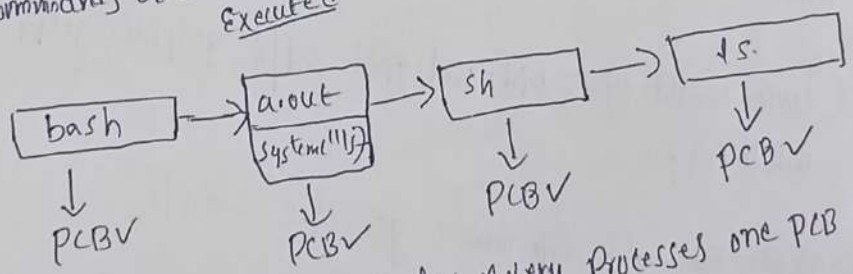
→ whenever when we use scanf, after the scanf what ever the statements present below they will execute?

Ans). when user supply the input, After that only it will execute..

Similarly whenever we are using a system() below the system function, whatever the statements are there → that statements only executed when that command → Eg PWD, ls → commands are completed which is passed in the system call.

Executed

Conclusion :-



→ Above mentioned all are processes, so for every processes one PCB is created.

in above sh is a shell like a bash.

system("ls"); → this command "ls" is executed under "sh" shell. → 16:40

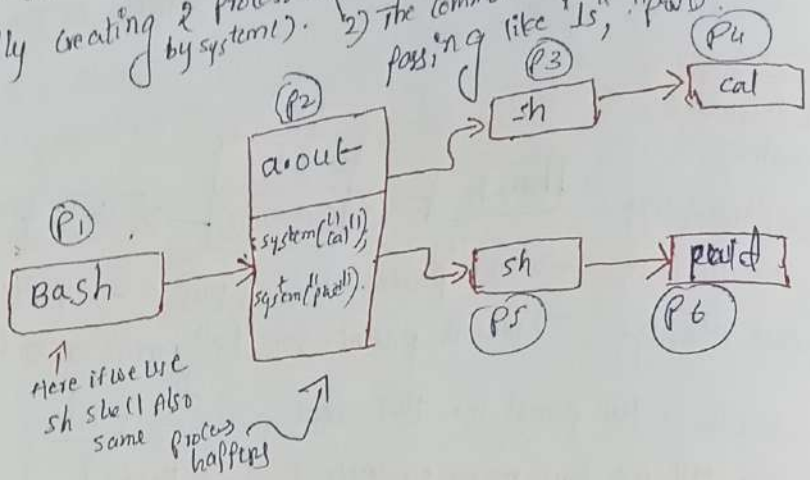
Question? why we need a 2 shells. ? "means" in above information.

why sh is creating already one shell is there (bash).

Ans) The Behaviour of system() is, when ever we call, system() call by default, internally creating 2 processes by system(). 1) sh 2) The command which we are passing like "ls", "pwd".

Example code :-

```
#include<stdio.h>
main()
{
Printf("Hello\n");
System("cal");
System("pwd");
Printf("Hi");
}
```



* In this program, we are calling the system() two times.

Execution flow :- first Hello is printed, After printing the hello, → system("cal"). So, bcz of this system call → System("cal") → one sh created → under sh → cal is created.

→ After "sh" is completed, After "cal" is completed, ~~system() will~~ next system() will be Executed.

→ In above program 6 processes involvement is there, as shown in the figure above.

Interesting discussion :- with Example Code :-

P1.c #include<stdio.h> → use #include "header.h"

```
void main()
{
printf("Hello world p1.c pid=%d ppid=%d\n", getpid(), getppid());
while(1);
}
```

↳ Create a seperate Executable file now → gcc p1.c -o p1.

P2.c

```
#include"header.h"
void main()
{
pf("in P2.c pid=%d ppid=%d\n", getpid(), getppid());
System("./p1");
Pf("In P2.c Bye.....\n");
}
```

→ In above program it is very clear that, in P2.c → we are again executing complete P1 (in middle of the P2 program).

→ gcc P2.c -o P2.

⇒ P1 is an infinite because of while(1);

→ Now run P2 program → ./P2

output → In P2.c pid = 228 Ppid = 8

Hello world P1.c pid = 230 Ppid = 229. → in P2 bye statement is not printed reason → simple guys beg P1 is in while(1) loop. (infinite). so the system function is blocking the process (dont this sentence) ⇒ got it → system(P1);

) now open the next terminal, & give the command ps -e

PID	TTY	TIME	CMD
1	?	00:00:00	init
7	tty1	00:00:00	init
8	tty1	00:00:01	bash
228	tty1	00:00:00	P2
229	tty1	00:00:00	sh
230	tty1	00:00:55	P1
231	tty2	00:00:00	init
232	tty2	00:00:00	bash
260	tty2	00:00:00	PS

for P2 → Parent is bash (Ppid 8) → for P2 → we are using system(), that's why 2 more processes are created internally → 1) sh → P1 PID is 230
2) P1

who is parent for P1 → 229 → who is 229 → sh.

who is parent for sh → our P2 process.

who is parent for P2 → ans → Bash.

Note :- In P2 program, what are the statements present after system() that are not going to executed. Because system() is blocking the process → BCZ of while(1); in P1 processes.

⊙ → when Ctrl-C executed in P2 program.?

when this command → system("./P1"); is completed or terminated then only Bye prints. (But that is not going to terminate normally because, we are while(1); so press control-C → So, when we press Ctrl-C while executing P2, P1 is terminated and printing "Bye" which is present in P2.

Q — when we press control C, what is happening ?

Bash → P2 system("./P1"); → sh → P1

waiting for user i/p , P2 (bye").

when we press control C → P1 is terminated, sh is terminated and the system will unblock the P1 process → so in P2 "Bye" statement is printed and P2 also get terminated. But Bash is waiting the user input.

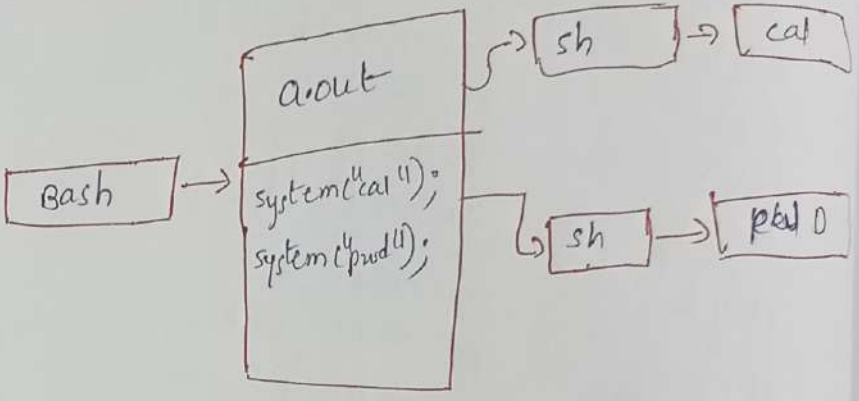
Will you agree "Bash" in most of the times it is in waiting state only. (Yes)

Example :-

@vector $ → now bash is waiting for command from user. → once user supplied command. eg "ls" → again the it (bash) waiting for that command to complete

Observe the below code & Diagram :-

```
#include<stdio.h>
int main()
{
printf("Hello\n");
system ("cal");
System ("pwd");
printf ("Hi");
}
```



Question ? In Above diagram, cal command and PWD Command are they Executed Concurently (or) Sequentially ?

Ans They are Executed sequentially, Because After total Completion of cal only pwd will Executes. [But still doubt how it is sequential Execution?

Conclusion :-

```
P1.c
main()
{
system("ls"); ⟩ These Processes are Sequential Execution only.
system("pwd"); ⟩ bcz if the one Process is Completed Completely, then only other Process is going to be Executed.
system("mkdir"); ⟩
}
```

→ space for above doubt clarification :- (may be Almost clear myself only).

* After getting the complete knowledge of linux system programming, we can implement / design our own shell like bash (or) sh.

*p- Error :-

```
Eg: #include "header.h"
    void main()
    {
    FILE *fp;
      fp= fopen ("data", "r");
      if (fp==0)
      {
      printf ("FILE not present.....\n");
        return;
      }
    printf ("File present\n");
    }
```

→ gcc → ls → files are listed & in that displayed files there is no file with "DATA".

→ ./a.out
↳ File not present.

Vi data (space) → "ABCDEF"
ls → now displayed → data ✓

(4 marks)

$ chmod u-r data
↑ chmod we will discuss in later classes so don't worry.

chmod is used for changing the permissions (file permissions).

now Again → ls → so displayed data also

→ But if we run the above program, it will display file not present.

why Even though file is present, then why f-open is failed there ?

why fopen returns zero in this case ?

Ans) Because, we are trying to open the file in read mode, but the file doesn't have the read permissions.

Conclusion :- Here fopen is failed, not bcz of data not present (off Course data is present) - fopen failed because it doesn't having permissions (read permission is not there).

*So there are so_many reasons, for fopen_failure

reason 1 file not present. present.

reason 2 file may present, but read permissions might have not there.

reason 3 hard_disk may be full (Explanation :- we already know in write mode, if file is not there, it will create a file, → but hard disk is full. then fail to create new file the fopen() return NULL.

* Some time in fopen() as first argument we need to supply the path, if we don't supply the path, it will considered as present working directory. (suppose you might have given wrong path there, suppose A file may not present, suppose the hard disk is full, suppose the permission is Denied, so there are so many reasons, that fopen() will fail. Reasons may be whatever, but incase of fopen() failure it returns Null. (0).

Note:- If you want to know, reason for the failure, don't use Printf, use perror.

↓ eg:-

```
#include<stdio.h>
void main()
{
FILE *fp;
fp=fopen("data","r");
  if (fp==0)
  {
   perror ("FOPEN");
    return;
  }
Printf ("File present\n");
}
```

↗ consider for this code we already changed file permissions, by using chmod.

→ gcc pl.c
→ ./a.out
O/P:
OPEN:- Permission Denied.

conclusion:- Better to use perror, instead of Printf, to display proper Error message.

→ Now delete data file. rm data
./a.out

Now: O/P
→ OPEN: No such file or directory,

* suppose if fopen() is success, but still, we are using perror → then perror will print success message.

Question: How P.Error knows that, fopen() is failed because of particular what reason?

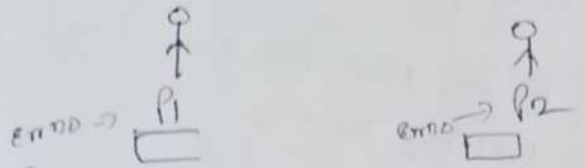
for that we need to understand one internal concept.

$ * man errno:- errno is a variable, for Every Process this variable will be there even though we didn't declare this. i,e errno.

we can check it in manpage → by giving the command, man errno.

Description:- The <errno.h> header file defines the integer variable errno, which is set by system calls and some library functions in the event of an Error to indicate what went wrong.

Explaination :- Consider there are two processes.



for Every Process one variable is created i,e errno.

for suppose, P1 Process is calling, fopen() then what fopen() will do 'means' upon failure, it will set one Error no into that particular process.

who will set this number ? :- that is set by system calls and some library functions in the Event of an Error.

How to chcke the system Errors ?

Any) → man Errno → Enter → scroll down, so that there we will found manuals (System Errors) like.

E2BIG : Argument list too long

EBADFD : File descriptor in bad state.

⋮

→ All these are the System Errors

EEXIST → file Exists (POSIX.1-2001).

EFAULT → Bad adress

EFBig → File too large

EHOSTDOWN → host is down

EACCESS → Permition is denied.

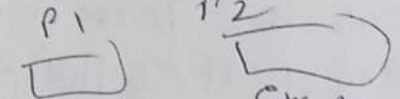
⋮

So all these are the system Errors.

* who set errno :-

* The function, which is failed, i,e only setting the Error number.

* for Every Process Error number will be there —



if P1 having fopen() & if fopen() is failed fopen set errono to P1 process so, Perror will look at this no. according to that number, it will maintain a look up table so, it will display message based on number.

Example Code :-

```
#include<stdio.h>
#include<errno.h>
int main()
{
Printf("%d\n", EFAULT);   14
Printf("%d\n", EACCESS);  13
Printf("%d\n", EFBIG);    27
}
```

o/p

Note :- must and should headerfile → #include<errorno.h> otherwise syntax error as EFAULT undeclared first use in this function.

Fixed - for every output.

Imp Point :- when we use a system(), that system() is a blocking function, BCZ → System("ls"); → until unless this statement completed, the statements below the system will not Execute.

Note :- So, the repeated functions, depend... internally, it will set this errno after particular process.

(All the above mentioned are Macros, we can print the numbers of that printing already done in the previous page).

note :- this perror will see that Error number, and it will display the Error message.

one interesting Point :- ⇒ man fopen → Enter scroll down until getting the RETURN VALUE.

RETURN VALUE :- upon successful completion fopen(), fdopen() and freopen() return a FILE pointer. Otherwise, NULL is returned and errno is set to indicate the error.

↳ similarly → open man system → Time also may be able to ~~find~~ Set the errno (system is setting error no or not we will able to find. → (But system() will not set that Error number).

So conclusion :- from now onwards, especially whenever we are writing the system programing, any function is failed, & if you want to know because of what reason that function is ~~failed~~, instead of writing printf("fail\n"), we need to use perror ~~function~~ to display Proper error message, (use perror immediately after the function call.

Note :- System Call 99.9% upon failure, it will return -1.

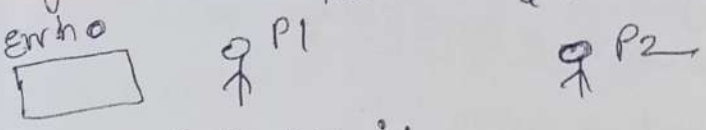
Eg :- man open → Enter [we know open is system call call by fopen() internally].

RETURN VALUE :- open(), openat(), and creat() return the new file descriptor, or -1 if an error occurred (in which case, errno is set appropriately).

Imp note :- Almost, All the System Calls will set the error number. But only few library functions will set the Error number for that we have to open man Page & check (for checking library fun. setting error no or not).

Note :- for debugging purpose, system() will helps a lot.

Eg :- Consider there are 2 processes.



incase, if P1 process is calling the fopen() so for this process errno is set.

where errno variable is stored ? → it is global variable, so stored in a data section.

Void perror (const char *s);

* The perror() function produces a message on standard error describing the last error encounted during a call to a system or library function.

* First (if s is not NULL and *s is not a null byte ('\0')), the argument string s is printed → (("lsd();"), followed by a colon and a blank. Then an error message corresponding to the current value of errno and a new line.

Note :- In the program, → main() { fork system() open system() perror("abcd();"); → So this perror displays only 2nd system() (open()) error only.

don't think system call from system calls like for(), open()

→ so to display 1st system call error also so that we use perror immediately, after the 1st system call only.

Question 4 :- in windows, we install ubuntu (in my lap) is it O.S → Ans Yes. But that O.S supports only commands, GUI will not be there in that O.S. means in that application, they given only Command inter-face. (only shells commands).

Space for p-error brief explanation in simple statements (missing some clarity)

Linux_13 :- Linux System programming

Topic :- Process Management (main topic)

** Fork() :- → Imp concept :-

* Fork() :- is one of the most important system call.

fork - creates a child process

* Fork is useful for creating a child process (or) duplicate proc

SYNOPSIS

#include <sys/types.h> → These 2 header files, we need to
#include <unistd.h> when we are using the fork() system call.

Prototype for fork() system call :-

pid_t fork (void);

Description :-

fork() creates a new process by duplicating the calling process.
The new process is reffered to as the child process.
The calling process is referred to as the parent process.

To understand above Description better we need to write one basic program

```
#include "header.h"
void main()
{
printf("Hello ----- \n");
printf("Hi ----- \n");
printf("pid = %d\n", getpid());
printf("pid = %d\n", getpid());
while(1);
}
```

→ gcc → a.out → output :-

Hello ---- Pid = 89
Hi Pid = 89

while(1) → is keep on Executing

Ans) process is running in fore ground or back ground

Ans) → Fore ground. so that is the reason, command prompt not came as an interface.

NOW, open another terminal & type ps -e

PID	TTY	TIME	CMD
1	?	00:00:00	init
7	tty1	00:00:00	init
8	tty1	00:00:00	bash
89	tty1	00:00:25	a.out

So, our process ID is 89, which is running in the terminal one → tty1
so, now close the terminal 2 And Again open the terminal one → now give control ^c → to terminate the process (incase if you give control c in terminal 2, process is not going to terminate), now after terminating the process, Again open terminal 2 and give the command ps -e → now we can observe process 89 is Not Present (it is terminated Already).

Now write the same Above code, Just by Adding the fork()

Extra.?

Example:-

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
void main()
{ printf("Hello---- pid=%d\n", getpid());
  fork();
  Printf("Hai --- pid=%d\n", getpid());
  while(1);
}
```

→ gcc → ./a.out. → Enter.

Hello ---- pid = 128
Hai ---- pid = 128
Hai ---- pid = 129

Very interesting o/p → BCZ in our code only 1 hai statement is present, but here 2 hai statements are gets Printed.

Conclusion :- In Above output, when hi is printed 2 times means both are different processes, that's why PID's are also different. but here we loaded a.out only one time. for a.out Parent is bash

bash → a.out → a.out is having 4 statements.

① → Pf("Hello----);
② fork();
③ Pf("Hai ---- pid=%d\n", getpid());
④ while(1);

Imp Point :- when fork() → duplicate/child. is called duplicate aout is created

Imp Point 2 :- After the fork(), whatever the statements are present, those statements are executed by parent and child.

Q After the fork(), how many statements are there? → Ans → 2 statements.
1) hi
2) while(1)

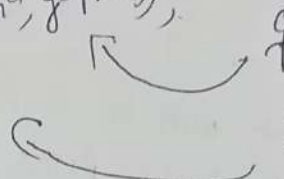
17min

so, both the parent process and child process executing those two statements which are present after the fork() system call.
Now, both child and parent processes are changing its state to Run to Ready, Read to Run, Run to Ready like this.

Question? why the hello is not printed by the child in above program?
Ans: when the printf("Hello\n"); is executed, in the parent process at that time child not yet created.
will you Agree, our C-program is sequential execution? -> Yes.
Explaination:- Because, after executing one statement only next statements are going to be executed.

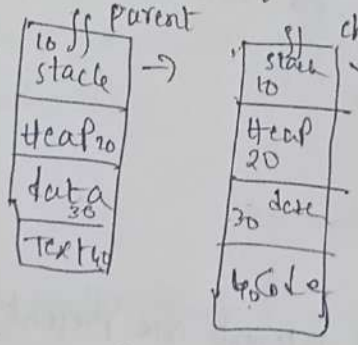
Above code explaination with respect to fork() :-

```
#include<stdio.h>
void main() -> PID = 50 (parent)
{
    printf("Hello ... pid = %d\n", getpid());   -> Parent only Executing this statement child not yet created.
    fork(); -> Parent only executing this statement at the time of child creation.
                -> now child created -> PID = 51
    printf("Hai ... pid = %d\n", getpid());
    while(1);
}
```

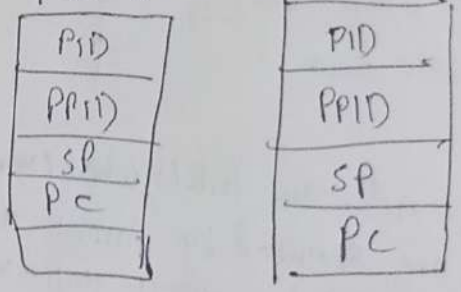


so now, both Parent and child going to Executes, the 2 statements.

Imp Point :- whenever a process is calling, the fork(), Everything is Duplicated Means -> our process is having -> stack, heap, data & code section ->



-> duplicate of Parent. PCB of Parent also duplicated to child.



Question 1

In Parent process before fork() int a=10; will be there, is that a=10 present in child stack -> yes. (duplicate of parent, stack, heap, data and Text) That's why.

So, now open another terminal → Type ps -e, so that we will observe. (25:40 min)

tty1 a.out → so we can observe 2 a.out's (BCZ of fork).

tty1 a.out

* Now write one more fork() in the above code.

Example :

```
1 #include "header.h"
2 int main()
3 {
4 printf("Hello ..... Pid = %d\n", getpid());
5   fork(); → y+1 → → → 6, 7 8
6   fork(); → y+2          ↓ Z
7 printf("Hai ---- pid = %d\n", getpid());
8   while(1);
9 }
```

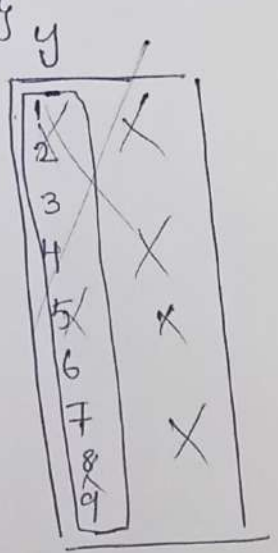
gcc p1.c -o y

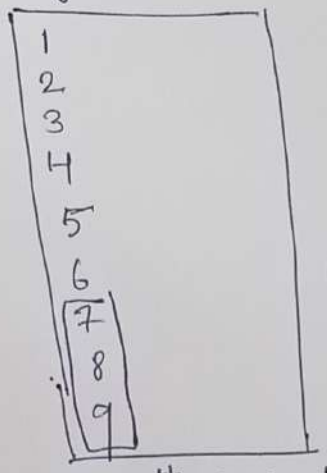
→ gcc → a.out (y)

y → printing Hello & executes fork. So one duplicate is created. → Consider it as y+1.

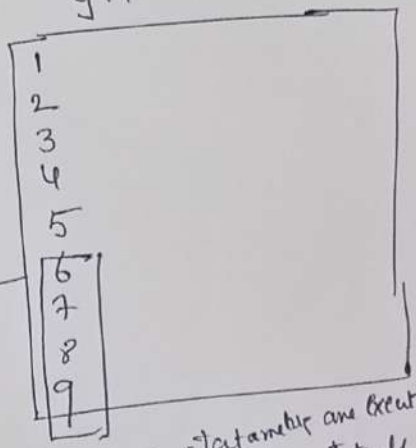
y+1 will executes 6, 7, 8 and 9.

→ Because of one more fork() in may process, one more child is created, Consider it as y+2.

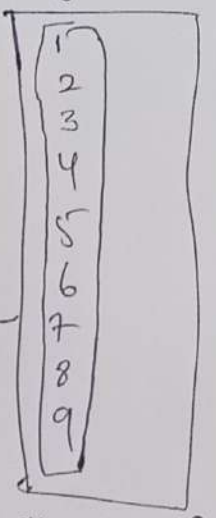




7, 8, 9th statements are Executed. (But having all the statements).



6, 7, 8 9 statements are executed but it is having all the statements to



All statements are Executed.

* very simple point: when a time slice is given to y+1 → So this y+1 process also need to call fork() → BCZ of that fork one du